

Temporal Counterfactual Explanations of Behaviour Tree Decisions

Anonymous Authors^a

^aAnonymous Affiliation,

ARTICLE INFO

Keywords:

explainability
interpretability
transparency
counterfactual explanation
causal explanation
behavior tree

ABSTRACT

Explainability is a critical tool in helping stakeholders understand robots, in particular, the ability for robots to explain why they have made a decision or behaved in a certain way. Behaviour trees are a popular framework for controlling the decision-making of robots and other software systems, and thus a natural question to ask is whether or not a system driven by a behaviour tree is capable of answering “why” questions. While explainability for behaviour trees has seen some prior attention, no existing methods are capable of generating causal, counterfactual explanations which detail the reasons for robot decisions and behaviour. Therefore, in this work, we introduce a novel approach which automatically generates counterfactual explanations in response to contrastive “why” questions. Our method achieves this by first automatically building a causal model from the structure of the behaviour tree as well as domain knowledge about the state and individual behaviour tree nodes. The resultant causal model is then queried and searched to find a set of diverse counterfactual explanations. We demonstrate that our approach is able to correctly explain the behaviour of a wide range of behaviour tree structures and states in real time, unlike previous methods which are either unable to answer contrastive questions with causal explanations, or are not guaranteed to provide consistent and accurate explanations. By being able to answer a wide range of causal queries, our approach represents a step towards more transparent, understandable, and ultimately safe and trustworthy robotic systems.

1. Introduction

Explainability is an important concern in the deployment of robots in the real world, especially when these robots operate with and around humans (Sado, Loo, Liew, Kerzel and Wermter, 2023). As robots are endowed with more complex decision-making, become more autonomous, and integrate further into people’s daily lives, it becomes critical that those who interact with the robot or are affected by its decision-making understand how the robot behaves. In particular, a robot should be able to explain the causal reasons why it made a particular decision, or why something unexpected, such as a failure, occurred. In doing so, not only can users better understand the robot, but they can also more safely and efficiently integrate it into real-world applications (Miller, 2019). Indeed, making robots and other artificial intelligence (AI) systems more understandable through explainability is core to the creation of responsible and safe AI systems (Herrera-Poyatos, Del Ser, de Prado, Wang et al., 2025; Walker, Haase, Mehalick, Steele et al., 2025).

In real-world decision-making tasks, many robots rely on structured control architectures. Behaviour trees (BTs) are a popular option for such an architecture, prevalent in robotics and across other fields such as artificial intelligence and computer gaming (Gugliermo, Domínguez, Iannotta, Stoyanov and Schaffernicht, 2024). In addition to their other qualities, such as reactivity and modularity, BTs are widely seen as an inherently interpretable, human-readable decision-making architecture (Iovino, Scukins, Styrod, Ögren and Smith, 2022), marking them as a promising tool in the design and implementation of transparent, interpretable robots.

However, the inherent interpretability of BTs may not be sufficient for improving understandability, especially for non-expert users who may not be able to easily interpret representations of BTs. Thus, the problem of generating explanations from BTs has seen some recent attention (Han, Giger, Allspaw, Lee, Admoni and Yanco, 2021; Ögren and Alfredson, 2023; Barkouki, Chuang and Robinson, 2024; LeMasurier, Gautam, Han, Crandall and Yanco, 2024a; Tagliamonte, Maccaline, LeMasurier and Yanco, 2024). Despite these efforts, the causal aspect of explainability has not been considered for BTs, and these approaches are unable to identify the reasons why a robot did or did not make a particular decision. A BT encodes a complex sequential relationship between BT nodes, and every state variable in the robot’s environment may affect or be affected by a BT node. These causal relationships should be explicitly modelled to provide correct, causal explanations for BTs. Performing counterfactual reasoning (reasoning about hypothetical scenarios to contextualise real events) over such a model is considered an essential component of answering causal

ORCID(s):

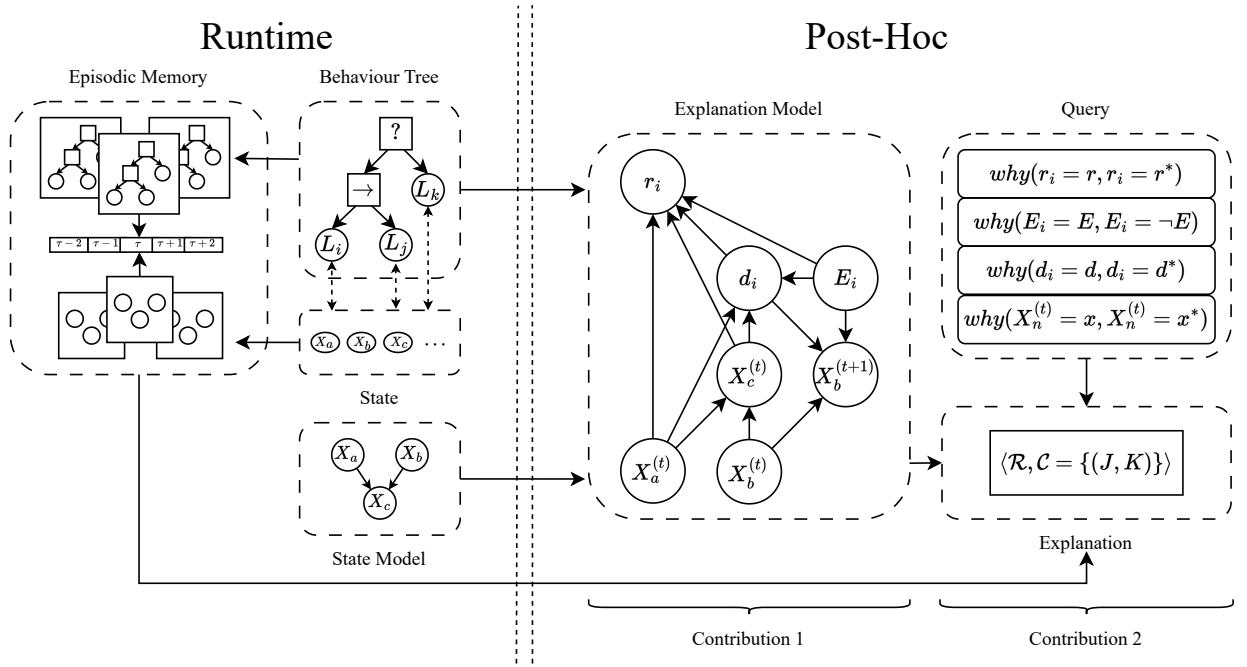


Figure 1: A diagram depicting the proposed architecture for explaining BT execution. At runtime, snapshots of the BT and the state are saved in an episodic memory (Section 5.1). Our contributions are twofold. The first contribution is an algorithm that builds a causal model (the explanation model) from the structure of the BT and state and decision-making knowledge (Section 5.2). Our second contribution is an algorithm to automatically generate explanations in real time by conducting a counterfactual search over the explanation model in order to satisfy a given query (Section 5.3).

“why” questions (Pearl and Mackenzie, 2018). Additionally, domain knowledge, where available, can be used to expand these causal models. This knowledge can come in the form of models of the robot’s environment, or prior knowledge about low-level decision-making components (such as BT nodes). To date, we are aware of no methods for generating causal explanations from BTs, except for generative (i.e. LLM-based) approaches, which are not guaranteed to correctly explain a particular decision (Xu, Jain and Kankanhalli, 2024). To ensure faithful understanding of the robot, provide safety assurances, and foster trust in the robot, it is essential that explanations are correct and respect the causal dynamics of the robot’s decision-making.

Thus, the problem of providing correct, causal explanations of BTs presents a clear research gap. To address this gap, we propose a novel architecture (depicted in Figure 1) that combines domain knowledge with the BT structure to create a causal model representation that enables the automatic generation of explanations in response to contrastive counterfactual queries. Our approach is capable of answering a wide range of queries regarding the success or failure of nodes in a BT, why a particular node was executed or not, the decisions the BT makes, and changes made to the state as a result of BT execution. Additionally, by leveraging an episodic memory that records past states and actions, our method can generate explanations based not only on the current state but also on previous states — both within a single BT tick and across longer temporal spans — supporting interactive follow-up questions. Our approach lays the foundation for behaviour tree-driven robots capable of interactively and automatically explaining the reasons for their decisions and behaviour.

In summary, the contributions of this work are as follows:

1. A method for combining structural and domain knowledge to create a causal model representation of the BT and the environment it interacts with (Algorithm 1).
2. An algorithm for conducting a counterfactual search over the generated causal model to produce a set of explanations in response to a contrastive query (Algorithm 6).

We evaluate our approach on a number of simulated behaviour trees (Section 7) to demonstrate that the method is able to produce correct explanations in real time. A comparison with an LLM-based approach (Section 7.3) highlights

the improvements our method offers in terms of explanation correctness and runtime. To ensure reproducibility, an implementation is provided at: <https://anonymous.4open.science/r/btcm-AFE7/README.md>.

2. Background

In this section, we provide some background on important concepts used in this work, namely behaviour trees (Section 2.1) and causal models (Section 2.2).

2.1. Behaviour Trees

A BT is a directed tree consisting of internal control nodes and leaf execution nodes (Colledanchise and Ögren, 2018). The tree functions by regularly signalling the root of the tree, which passes this signal (known as a *tick*) to its children according to the type of node. Upon receiving a tick, a node is executed and returns either *Success* (indicating the node's function has been executed successfully), *Failure* (indicating the node's function has failed in some way), or *Running* (indicating the node's function is still executing). It can also be useful to regard nodes that have not been ticked as returning an *Invalid* (or *Idle*) status until such time that they are executed (Colledanchise and Natale, 2021).

Leaf nodes can be categorised conceptually as either *Action* nodes, which execute a command, or *Condition* nodes, which check for a particular condition. Action nodes return *Success* if their command is executed successfully, *Failure* if the command fails, and *Running* if the command is still executing. Condition nodes, on the other hand, return *Success* if the condition is met and *Failure* otherwise, never returning *Running*.

There are multiple commonly-used control nodes, of which we consider the two most fundamental - *Sequence* and *Fallback* nodes. *Sequence* nodes, denoted using the $\rightarrow$ symbol, have two or more children which are executed in sequence, from left to right. The sequence node returns *Success* if all of its children return *Success*, returning *Failure/Running* if a single child returns *Failure/Running*.

Fallback nodes, also known as *Selector* nodes, denoted using the $?$ symbol, have two or more children which are executed in sequence as with sequence nodes. However, the fallback node returns *Failure* only if all of its children return *Failure*, otherwise returning *Success/Running* if a single child returns *Success/Running*.

Following Colledanchise and Ögren (2014), we can formally define a behaviour (sub)tree as the tuple $\mathcal{T}_i = \langle f_i, r_i, \Delta t \rangle$, where

- Δt is the time difference between ticking the tree and receiving an output.
- $f_i : \mathcal{S} \rightarrow \mathcal{S}$ is a function which determines the effect the activation of the subtree has on the state given an input state. In other words, $s^{(t_{k+1})} = f_i(s^{(t_k)})$, where $t^{(k+1)} = t^{(k)} + \Delta t$ and k is a time step index. Here $\mathcal{S}$ is the set of all states and $s \in \mathcal{S}$ (see Section 4.1).
- $r_i : \mathcal{S} \rightarrow \{\text{Running}, \text{Success}, \text{Failure}, \text{Invalid}\}$ denotes the return function of the tree.

As nodes are organised in a tree structure, we can denote the set of children of $\mathcal{T}_i$ as $Ch(\mathcal{T}_i)$, and order all nodes in the tree such that $\mathcal{T}_i < \mathcal{T}_k$ indicates that $\mathcal{T}_i$ is to the left of $\mathcal{T}_k$. Similarly, $Pa(\mathcal{T}_i)$ denotes the parent node of $\mathcal{T}_i$. If two nodes share the same parent, they are referred to as siblings. As nodes are ordered from left to right, we call the child $\mathcal{T}_j \in Ch(\mathcal{T}_i)$ the “left-most” child of $\mathcal{T}_i$ if $\nexists \mathcal{T}_c \in Ch(\mathcal{T}_i)$ such that $\mathcal{T}_c < \mathcal{T}_j$.

2.2. Causal Models

Causality is considered a central concept to explaining *why* an event occurs, and modelling the causal relationships in a system with a *causal model* is an important step in providing causal explanations (Pearl and Mackenzie, 2018). Graphical causal models are particularly useful and represent causal relationships as edges in a directed acyclic graph (DAG). For the purposes of this work, we can define a causal model $\mathcal{M}$ as a tuple $\mathcal{M} = (\mathcal{G}, \mathcal{R}, \mathcal{F})$, where $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ a DAG with nodes in $\mathcal{V}$ and edges in $\mathcal{E}$, $\mathcal{R}$ defines the range of each variable in $\mathcal{V}$, and $\mathcal{F}$ is a set of functions, one for each variable in $V \in \mathcal{V}$, mapping a product of all parents of V in $\mathcal{G}$ to a value in $\mathcal{R}(V)$ (Miller, 2021).

An important operator used in querying causal models is the *Do* operator, which represents a causal intervention. The intervention $Do(X = x)$ asks the question “What would happen if I force $X = x$ to be true regardless of the causal influences on X ” (Pearl and Mackenzie, 2018). When applied to a model $\mathcal{M}$, $Do(X = x)$ deletes all incoming edges to X and forces the value of the variable to x . The values of the causal descendants of X can then be calculated using the existing values in the graph and the functions in $\mathcal{F}$.

3. Related Work

The problem of automatically generating explanations for robots has seen some attention, adopting a wide variety of approaches from the field of explainable artificial intelligence. Some methods, such as that of Gavriilidis, Munafo, Pang and Hastie (2023), learn interpretable surrogate models such as decision trees that approximate a robot's behaviour. Other methods leverage reinforcement learning, such as Cruz, Dazeley, Vamplew and Moreira (2023), who explain a robot's learned policy in terms of task success probability, Angelopoulos, Mangiacapra, Rossi, Di Napoli and Rossi (2025), who incorporate user preferences in a reinforcement learning policy, and generate explanations when these preferences are violated. In this work, we focus specifically on counterfactual explanations, which identify important factors for a decision by perturbing an input in order to determine what changes would result in different decisions (Guidotti, 2022). Counterfactual reasoning has been argued to be an essential component of answering "why" questions whose answers leverage the causal reasons for a given decision or event (Miller, 2019; Pearl and Mackenzie, 2018). Counterfactual methods are related to perturbation-based approaches, although the focus is typically on finding minimal changes and root causes, rather than on finding important features (Gómez-Talal, Azizsoltani, Bote-Curiel, Rojo-Álvarez and Singh, 2025).

Counterfactual explanations have seen some application in robotics domains. For example, Gjærum, Strümke, Lekkas and Miller (2023) learn model trees as surrogates for a robot's decision-making and use them to generate counterfactual explanations. However, by not explicitly modelling causal assumptions on the state and decision-making, such a method may produce infeasible counterfactual scenarios that require additional feature engineering or optimisation constraints. Additionally, by not explicitly modelling causal relationships between features, an explanation might wrongly attribute a cause to a variable which is merely correlated to a true cause. Some methods counteract these problems by explicitly leveraging causal models when generating counterfactuals. For example, Diehl and Ramirez-Amaro (2022) learn a causal Bayesian network in simulation and then perform a breadth first search over the model to find valid counterfactuals, while Love, Andriella and Alenyà (2024) construct a simple causal graph modelling a human-robot interaction (HRI) scenario and generate sets of explanations by systematically intervening on nodes in the graph. However, these approaches do not consider the effects of prior states and decisions on the target failure or decision respectively, except where such history is encoded in the current state. In an extension to their previous work, Diehl and Ramirez-Amaro (2023) account for temporal sequences of states and decisions by considering snapshots of states at discrete time intervals when learning the structure of the causal Bayesian network. Nevertheless, this method remains focused on explaining failures rather than the reasons for decisions and states. It also requires learning a causal model in simulation, without exploiting the structure of the robot's control architecture to automatically define causal relationships in the robot's behaviour.

As our work specifically leverages the structure of behaviour trees in order to construct a causal model and generate counterfactual explanations, we now discuss some existing approaches linking explainability and BTs. BTs are considered to be inherently more interpretable than black box models such as neural networks, and thus several approaches focus on learning BTs as surrogate models (Potteiger and Koutsoukos, 2023; Wan, Liu, Liu and Xu, 2024). While this claim is certainly true, and much can be understood by examining depictions of a BT state, it is nevertheless useful to explicitly generate explanations from BTs, especially for non-expert users. There are some works in this area, such as the foundational work of Han et al. (2021), who introduce several algorithms that are capable of answering a variety of questions about a behaviour tree at runtime, such as "What are you doing?", answered by describing the current node in execution, "Why are you doing this?", answered by describing ancestors of the current node in the tree, and "What is your (sub)goal?", answered by describing the entire (sub)tree. Of these questions, only "Why are you doing this?" is causal in nature, and then only querying the execution of the currently executing BT node, answering by referring to the execution of a single ancestor node. This approach is not designed to cater to a wider variety of causal queries targeting decisions, node return statuses or state variable values. It is also not designed to give causal explanations for events that did not occur (e.g. answering "Why didn't you do that?").

The approach by Han et al. (2021) is expanded on by Barkouki et al. (2024), who generate explanations by predicting future actions using the structure of a BT, answering questions such as "What will you do next if the current action succeeds?". Ögren and Alfredson (2023) extend the work to backward chained BTs, producing explanations that reference both successful and unsuccessful goals as well as a trace of node executions up the tree from the currently executing BT node. By constructing a BT as a "Make-sure BT", condition nodes which check action preconditions allow for state variables references by preconditions to be included in the explanation. LeMasurier et al. (2024a) also expand on the work of Han et al. (2021) by introducing assumption checker nodes to the BT, which track whether

or not key assumptions have been violated. If such assumptions have been violated, the robot can predict upcoming failures and proactively explain them. While these approaches are indeed useful, they do not address the causes of decisions and behaviour beyond the sequence of subtree executions (and, in the case of Ögren and Alfredson (2023), certain state variables representing preconditions). For an illustrative comparison between our method and those of Han et al. (2021) and Ögren and Alfredson (2023), see Section 6.2.

Recently, LLMs have been presented as a tool for both generating and formatting explanations of robotics systems. For example, Frering, Steinbauer-Wagner and Holzinger (2025) allow users to interface with an LLM which can produce explanations for the behaviour of a belief-desire-intention (BDI) agent in response to natural language queries. The explanation prompt includes information on the BDI agent’s “mind state” and the interaction history of the system with the user. Gebellí, Hriscu, Ros, Lemaignan, Sanfeliu and Garrell (2025) provide an LLM with both static (e.g. behaviour rules) and dynamic knowledge (e.g. execution logs), as well as a user profile, in order to generate personalised natural language explanations. Of particular relevance to our work is that of Tagliamonte et al. (2024), who provide a general framework for answering user queries about a BT using an LLM, providing information such as the BT (encoded in YAML format), its current state, and any violated assumptions in the form of a prompt. Such an approach is clearly attractive, as both queries and explanations can be provided in natural language. However, there are no guarantees that the explanation provided by an LLM corresponds to the reasoning of the BT. Furthermore, results in a follow-up user study found the generated explanations to be less understandable and trustworthy compared to a templated approach (LeMasurier, Tagliamonte, Breen, Maccaline and Yanco, 2024b). In contrast, our approach explicitly incorporates the structure of the BT and domain knowledge about tree nodes and the state in order to provide explanations that are faithful to the BT’s reasoning and the dynamics of the environment, as demonstrated in Section 7.

Of all the methods presented in this section, only the LLM-based approaches are capable of responding to the same types of contrastive queries that our method targets, due to their ability to process arbitrary textual inputs to produce coherent natural language outputs. We therefore directly compare our method to an end-to-end LLM-based approach, representative of this family of approaches, in Section 7.3.

4. Definitions

In this section, we present the formalism we use to specify counterfactual queries and explanations (Section 4.2), as well as our extension of the formal definition of a behaviour tree (Section 4.3).

4.1. Decision-Making

Decision-making problems are often expressed in terms of states, configurations of the environment, actions, and decisions taken by the system (Russell and Norvig, 2009). In this work, we assume that the set of all states, termed the *state space* and denoted S , can be factored into a configuration of state variables $S = \prod_{n=1}^{|\mathcal{X}|} X_n$, where $\mathcal{X}$ is the set of state variables and $X_n \in \mathcal{X}$. Thus, each state can be expressed as $s = \langle x_1, \dots, x_n \rangle$, where each x_n denotes a particular value of X_n . Similarly, we term the set of all actions the *action space*, denoted $\mathcal{A}$. We assume that the null action $\emptyset$ - that is, the system doing nothing - is a member of $\mathcal{A}$.

4.2. Counterfactual Queries and Explanations

In this work, we seek to answer questions of the form “Why A and not B ?”. Following Miller (2021), we assume that the true event A (the *fact*) and untrue event B (the *foil*) refer to the same variable or set of variables, and that they are mutually exclusive (i.e. they cannot both be true). We denote the query using the notation $why(A, B)$.

To answer such a query, we define a counterfactual explanation as a tuple $\langle \mathbf{R}, \mathbf{C} \rangle$. $\mathbf{R}$ denotes the *reason set*, which is a set of true statements or *reasons* that each explain the fact A . $\mathbf{C}$ denotes the *counterfactual set*, which is a set of counterfactual tuples $\langle \mathbf{J}, \mathbf{K} \rangle$, where $\mathbf{J}$ is a statement that, if true, implies $\mathbf{K}$ is true, where $\mathbf{K} \implies B$. Each $\langle \mathbf{J}, \mathbf{K} \rangle \in \mathbf{C}$ must correspond to a reason in $\mathbf{R}$ (and inversely, each reason should have at least one corresponding counterfactual), such that $\mathbf{J}$ is mutually exclusive with the reason.

As an example, consider a robot running a cognitive exercise, in which a user is shown a sequence of symbols by the robot and must repeat the sequence from memory (this use case, known as the *serial recall* task, is properly introduced in Section 7.2). Suppose that, after receiving an incorrect sequence from the user, the robot decides to end the task. A caregiver reviewing the interaction might ask “Why did the robot end the task rather than repeat the sequence for the user?”. As a formal query, this can be posed as $why(fact : d = EndTask, foil : d = RepeatSequence)$. Suppose that the robot provides the formal explanation $\langle \mathbf{R} : UserFrustration = 0.8, \mathbf{C} : \langle \mathbf{J} : UserFrustration \leq 0.7, \mathbf{K} :$

$d = \text{RepeatSequence})$). This explanation would, translated to natural language, read as: “The robot ended the current task because the user’s frustration level was 0.8. If the user’s frustration level was 0.7 or lower, the robot would have repeated the sequence for the user.”

4.3. Extending Behaviour Trees

While the formal representation of BTs presented in Section 2.1 encodes the sequence of decisions made by the system, it does not in itself capture the relationships between the concepts involved in the decisions, such as which parts of the state are directly related to particular parts of the decision-making architecture. Thus, to obtain a rich set of counterfactual explanations from a BT which also incorporates this domain knowledge about the relationship between the state and the decision-making (e.g. the input/output of each node), we extend the definition of Colledanchise and Ögren (2014) within the framework of the decision-making problem outlined in Section 4.1. In particular, we define a leaf node L_i , an extension of the generic subtree $\mathcal{T}_i$, as a tuple $\langle \mathcal{X}_i, \mathcal{Y}_i, \mathcal{A}_i, f_i, p_i, d_i, r_i, \Delta t \rangle$, whose elements are defined as follows:

- $\mathcal{X}_i \subseteq \mathcal{X}$ denotes the set of state variables taken as input by the leaf node (the input variables). In practice, this set could represent blackboard variables read by the node. From $\mathcal{X}_i$, we can define an input state space $\mathcal{I}_i = \prod_{n=1}^{|\mathcal{X}_i|} \mathcal{X}_n$.
- $\mathcal{Y}_i \subseteq \mathcal{X}$ denotes the set of state variables whose values can be directly changed by the leaf node (the output variables). In practice, this set could represent the blackboard variables written to by the node. From $\mathcal{Y}_i$, we can define an output state space $\mathcal{O}_i = \prod_{n=1}^{|\mathcal{Y}_i|} \mathcal{Y}_n$.
- $\mathcal{A}_i \subseteq \mathcal{A}$ denotes the set of actions that can be selected by the node. We assert that the null action $\emptyset \in \mathcal{A}_i$.
- $p_i : \mathcal{I}_i \rightarrow \mathcal{O}_i$ represents the direct effect of the nodes’ execution on the output variables in $\mathcal{Y}_i$, given the input variables in $\mathcal{X}_i$.
- $d_i : \mathcal{I}_i \rightarrow \mathcal{A}_i$ denotes the decision-making function (i.e. policy) of the node, determining which action is selected when the node is executed. If the node is not executed, or if the node cannot select an action (such as with Condition nodes), then $d_i = \emptyset$.
- f_i, r_i and Δt retain their definitions from Section 2.1.

For convenience, we can also define the statement E_i , which is true if L_i is executed and false otherwise.

We motivate our decision to include $\mathcal{A}_i$ and d_i in this extended definition by noting that querying the behaviour and actions taken by a robot is an important aspect of explainable robotics (Wachowiak, Fenn, Kamran, Coles, Celiktutan and Canal, 2024). If a node or subtree can only ever execute one action primitive, then this addition is redundant. However, if a node is capable of executing more than one action, our formulation is required in order to explain why one action was executed instead of another (see Section 5.3). This distinction is also useful for differentiating between intentions (decisions) and executions, where the actual execution of a decision may fail.

Here we also note an important assumption we make going forward: that the functions p_i , d_i , and r_i are deterministic. This assumption is necessary for the counterfactual search we present in Section 5.3 (Algorithm 6), as it repeatedly executes p_i , d_i , and r_i , which must return consistent results. Similarly, f_i should be deterministic in order to ensure that the explanations are consistent with the real environment. For a discussion on this limitation and on future work arising from it, see Section 8.

5. Architecture

Having provided the necessary definitions, we now present our methodology to generate explanations for systems (such as robots) whose decision-making is governed by a BT. Our architecture, depicted in Figure 1, relies on episodic memory stored at runtime (Section 5.1). It operates by constructing a graphical model (explanation model) that during or post-execution captures the causal relationships derived from both the BT structure and domain knowledge about the state and decision-making (Section 5.2). Using both the explanation model and the episodic memory, our method then performs a counterfactual search to automatically generate explanations in real time that satisfy a given query (Section 5.3). A high-level flowchart depicting this process is depicted in Figure 2. In addition to presenting the algorithms that make up the core contributions of this work, we also discuss some properties thereof and conduct complexity analyses in Section 5.4.

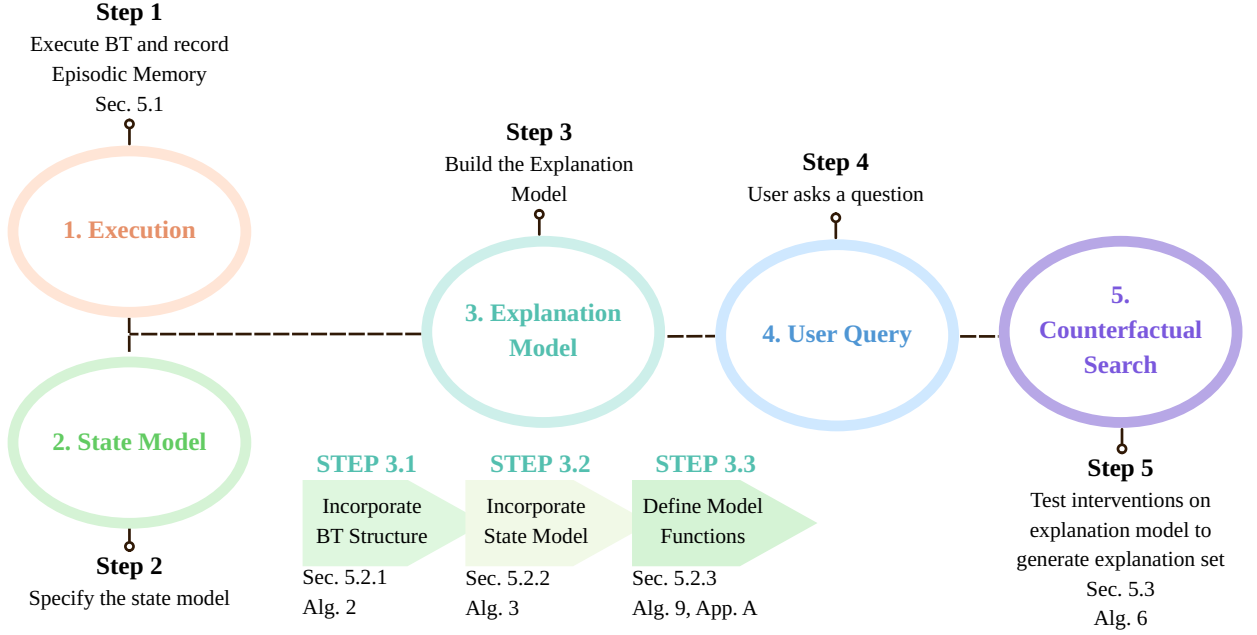


Figure 2: A high-level overview of the implementation of our approach. The first two steps, which may occur in any order, are to 1) execute the BT and record the episodic memory in a suitable format, and 2) specify the domain knowledge to be injected into the system in the form of a state model $\mathcal{M}_S$. Subsequently, our framework 3) builds an explanation model, which is a causal model representing the BT and the state which can be used to generate explanations. Constructing the model contains three substeps, namely 3.1) incorporating the BT structure into the model structure, 3.2) incorporating the state model into the model structure, and 3.3) defining variable ranges and functions for the model. Finally, whenever 4) a user asks a contrastive question (Section 4.2), our framework 5) conducts a counterfactual search over the explanation model to generate a set of explanations that answer the user's query.

5.1. Execution

Episodic memory - a detailed memory of states, actions and other relevant information over time - is considered an essential component to explaining why a particular event, such as a robot decision or BT state, occurs (Langley, Meadows, Sridharan and Choi, 2017; DeChant, 2023).

Our method relies on the system maintaining such an episodic memory which captures snapshots of the BT and the values of state variables, in order to be able to reconstruct the context of a queried event and be able to consider counterfactual scenarios deriving from those circumstances. In particular, we record all changes to the state variables in $\mathcal{X}$, as well as the return status (and action, if applicable) of a node when it is executed. In this way, the discrete unit of time is defined by the execution of exactly one node.

All BTs in this work are implemented in the *py_trees* library (Stonier, Demir, Walker, Castro et al., 2025), with memories recorded by a custom visitor.

5.2. Building the Causal Model

In order to generate counterfactual explanations, our method requires a graphical model that encodes the causal relationships present between state variables in $\mathcal{X}$ and within the BT (between the execution E_i , return status r_i and decision d_i for each BT node $\mathcal{T}_i$). Therefore, one of the core contributions of this work is an algorithm to automatically construct such a causal model, which we term the *explanation model*.

Recall from Section 2.2 that such a model is specified by the tuple $M = (\mathcal{G}, \mathcal{R}, \mathcal{F})$. The graphical structure $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is derived from both the known structure of the BT (Section 5.2.1) as well as domain knowledge about the state and decision-making (Section 5.2.2), as represented in Algorithm 1 (a visual representation is also provided in Figure 8, Appendix A). Given this structure, we can then determine the ranges $\mathcal{R}$ and functions $\mathcal{F}$ for each node in the model (Section 5.2.3).

As a BT is a sequential decision-making structure whose nodes are executed at different points in time, the explanation model must be capable of representing a temporal sequence of events rather than a single moment in

Algorithm 1 Building the explanation graph

```

1: function BUILDExplanationGraph
2:    $\mathcal{G} \leftarrow \text{GRAPHFROMSTRUCTURE}()$  ▷ Alg. 2
3:    $\mathcal{G} \leftarrow \text{ADDDOMAINKNOWLEDGE}(\mathcal{G})$  ▷ Alg. 3
4:   return  $\mathcal{G}$ 
    
```

time. In our construction, the explanation model represents the system for the duration of a single, arbitrary “tick” of the BT (i.e. from the execution of the tree root to the moment it returns a valid return status).

5.2.1. Incorporating Behaviour Tree Structure

As mentioned before, some causal relationships in $\mathcal{G}$ can be determined purely from the structure of the BT. Variables can be inserted into the explanation model for each node in the tree. We devise different methods for relating these variables in the explanation graph, depending on the type of node and its relationship with its parent and siblings. The algorithm for determining the partial causal graph from the BT structure is presented in Algorithm 2 (a full version is given in Algorithm 7, Appendix A).

Algorithm 2 Constructing the explanation graph from the BT structure (full version in Algorithm 7, Appendix A)

```

1: function GRAPHFROMSTRUCTURE( )
2:    $\mathcal{G} \leftarrow \emptyset$ 
3:    $\mathcal{G} \leftarrow \text{ADDVARIABLESFROMBTNODES}(\mathcal{G})$  ▷ Insert graph nodes for each BT node
4:   for node  $\mathcal{T}_i$  in the BT do
5:     Add edge  $E_i \rightarrow r_i$  to  $\mathcal{E}$ 
6:     if  $\mathcal{T}_i$  is a leaf node then
7:        $\mathcal{G} \leftarrow \text{ADDLEAFEDGES}(\mathcal{G}, \mathcal{T}_i)$  ▷ Depicted in Figure 3a
8:     else
9:        $\mathcal{G} \leftarrow \text{ADDCOMPOSITEEDGES}(\mathcal{G}, \mathcal{T}_i)$  ▷ Depicted in Figure 3b
10:    if  $\mathcal{T}_i$  is not the tree root then
11:       $\mathcal{G} \leftarrow \text{ADDPARENTSIBLINGEDGES}(\mathcal{G}, \mathcal{T}_i)$  ▷ Depicted in Figures 3c and 3d
12:  return  $\mathcal{G}$ 
    
```

First of all, we note that for any node $\mathcal{T}_i$, it can only return a status in $\{\text{Running}, \text{Success}, \text{Failure}\}$ if it is executed, otherwise returning *Invalid*. Thus, r_i is dependent on E_i (Algorithm 2, line 5). For a leaf node L_i for which d_i is defined (i.e. an action node), we note some additional relationships (Algorithm 2, line 7, depicted in Figure 3a). The decision d_i is dependent on E_i , as $d_i = \emptyset$ by definition if the node is not executed. If the node’s decision-making is implemented in two stages, as we do in this work, such that a decision is first made and then acted out in the environment, then we can say that r_i depends on d_i . For implementations where this does not hold true, then this relationship may be ignored.

For a composite node $\mathcal{T}_i$, we note that r_i is dependent on the return statuses of its children $\mathcal{T}_{c_1}, \dots, \mathcal{T}_{c_N} \in \text{Ch}(\mathcal{T}_i)$ (Algorithm 2, line 9, depicted in Figure 3b), following the rules outlined in Section 2.1.

Finally, we consider the influences on whether or not a node $\mathcal{T}_i$ is executed (E_i). If $\mathcal{T}_i$ is the root of the tree, then we assume E_i to be true. Otherwise, the causal relationships depend on the position of $\mathcal{T}_i$ among its siblings (Algorithm 2, line 11, depicted in Figures 3c and 3d). Suppose $\mathcal{T}_p = \text{Pa}(\mathcal{T}_i)$. If $\mathcal{T}_i$ is the left-most child of $\mathcal{T}_p$, then for both sequence and fallback nodes, $\mathcal{T}_i$ is executed first and thus $E_i = E_p$ (Figure 3c). Otherwise, there must exist some sibling $\mathcal{T}_j$ immediately to the left of $\mathcal{T}_i$. We observe then that E_i is dependent on r_j , with the function governing that relationship depending on whether $\mathcal{T}_p$ is a sequence or fallback node (see Section 5.2.3), as these nodes execute their children from left to right with stopping conditions dependent on child return statuses (Figure 3d).

5.2.2. Incorporating Domain Knowledge

Once the partial explanation graph has been constructed from the known BT structure, further causal relationships can be inserted from domain knowledge. This process is depicted in Algorithm 3, and an example is provided in Figure 4. We distinguish between two types of domain knowledge: *state knowledge*, regarding the causal relationships

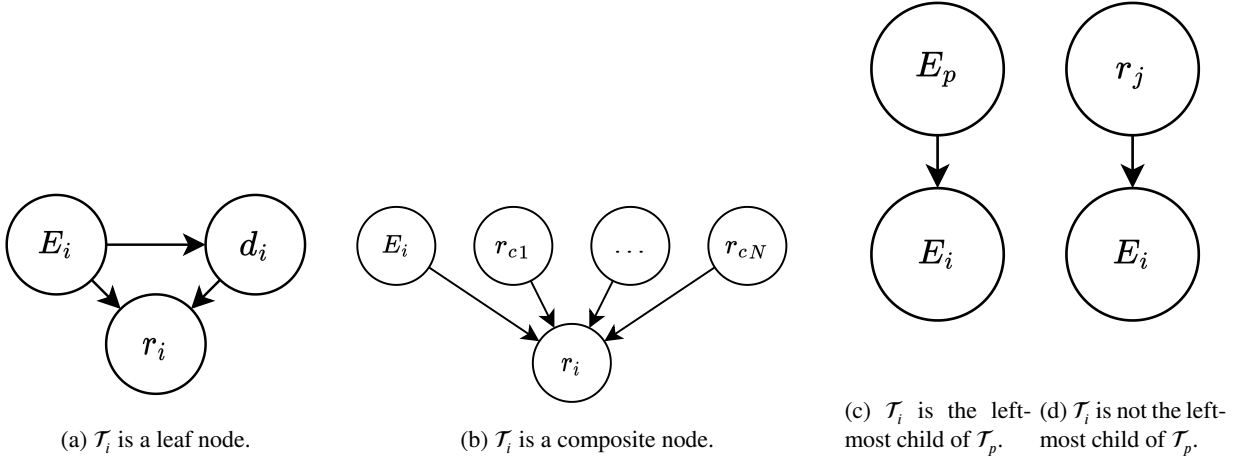


Figure 3: Explanation subgraphs of $\mathcal{T}_i$. For leaf nodes, E_i , d_i and r_i are related as in (a). For composite nodes, r_i is related to E_i and $r_{c1}, \dots, r_{cN}$ as in (b), where $\mathcal{T}_{c1}, \dots, \mathcal{T}_{cN} \in \text{Ch}(\mathcal{T}_i)$. When $\mathcal{T}_i$ is the left-most child of its parent $\mathcal{T}_p = \text{Pa}(\mathcal{T}_i)$, E_i is dependent on E_p , as in (c). Otherwise, E_i is dependent on r_j , as in (d), where $\mathcal{T}_j$ is the sibling of $\mathcal{T}_i$ immediately to its left.

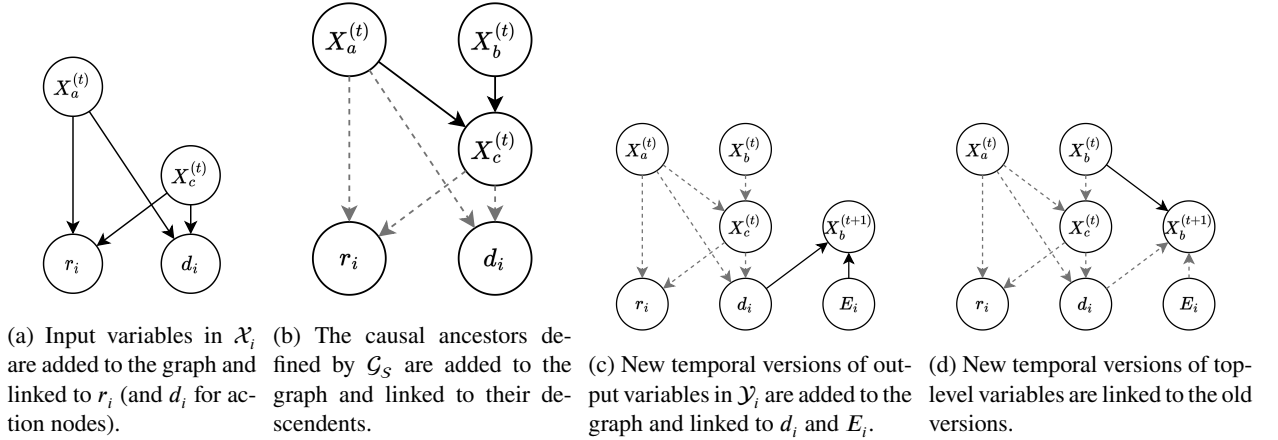


Figure 4: An example of the edges added by Algorithm 3 for one L_i . In this example, the state graph $\mathcal{G}_S$ is identical to the one in Figure 5, $\mathcal{X}_i = \{X_a, X_c\}$, and $\mathcal{Y}_i = \{X_b\}$.

between variables in $\mathcal{X}$, and *decision-making knowledge*, regarding elements of the decision-making process on the level of individual nodes in the BT.

State knowledge is represented by a state model $M_S = (\mathcal{G}_S, \mathcal{R}_S, \mathcal{F}_S)$. Inserting this knowledge into the explanation model allows for the generated explanations to include a wider range of causes (e.g. including not just an input variable but its causal ancestors as well) and to account for the delayed temporal effects of BT node executions and decisions (e.g. a decision impacting a state variable whose causal descendent subsequently impacts another event). An example state graph is provided in Figure 5.

Decision-making knowledge is represented by the extended BT sets and functions defined in Section 4.3. In particular, $\mathcal{X}_i$, $\mathcal{Y}_i$ and $\mathcal{A}_i$ must be provided. The functions p_i , d_i and r_i are not necessarily known, but can be evaluated for any given input by executing the BT node.

As the values of state variables can change over time due to the execution of the BT nodes (modelled by p_i), it is important to denote the value of each state variable at different times using different variables in the explanation model, similarly to other graphical models that consider a temporal aspect, such as dynamic Bayesian networks (Kanazawa, Koller and Russell, 1995). To facilitate this, we define a correspondence operator $\hat{=}$ such that $X^{(t)} \hat{=} X$ denotes that

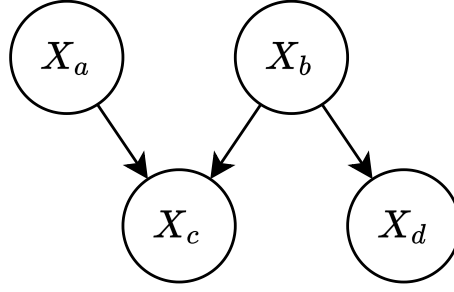


Figure 5: An example state graph with four variables in $\mathcal{X}$.

Algorithm 3 Completing the explanation graph with domain knowledge

```

1: function ADDDOMAINKNOWLEDGE( $\mathcal{G}$ )
2:   Define mapping  $\tau : \mathcal{X} \rightarrow \mathbb{Z}$ 
3:    $\forall X \in \mathcal{X}$ , add  $X^{(0)}$  to  $\mathcal{V}$  and set  $\tau(X) \leftarrow 0$ 
4:   Let  $\mathcal{L}$  be a list of leaf nodes sorted from left to right
5:   for  $L_i \in \mathcal{L}$  do
6:      $\mathcal{G}, \tau \leftarrow \text{ADDLEAFINPUTS}(\mathcal{G}, L_i, \tau)$  ▷ Algorithm 4
7:     if  $L_i$  is an action node then
8:        $\mathcal{G} \leftarrow \text{ADDLEAFOUTPUTS}(\mathcal{G}, L_i, \tau)$  ▷ Algorithm 5
9:      $\mathcal{G} \leftarrow \text{ADDTEMPORALEDGES}(\mathcal{G}, \tau)$  ▷ Algorithm 8 (Appendix A)
10:  return  $\mathcal{G}$ 
    
```

the variable $X^{(t)} \in \mathcal{V}$ represents a temporal version of $X \in \mathcal{X}$. To keep track of the temporal versions of each variable, we maintain a mapping $\tau : \mathcal{X} \rightarrow \mathbb{Z}^{0+}$ that maps each variable in $\mathcal{X}$ to its latest temporal index.

To begin, we add an initial version $X^{(0)}$ of each $X \in \mathcal{X}$, representing the state at the beginning of a “tick” (Algorithm 3, line 3). Then, for each leaf node L_i , in order from left to right in the BT, we perform two steps - adding incoming edges based on $\mathcal{X}_i$ and outgoing edges based on $\mathcal{Y}_i$.

In the first step, we add the latest version of X and all of the causal ancestors of X , as defined by τ , to the variable set $\mathcal{V}$ for $X \in \mathcal{X}_i$. We relate each of these variables to r_i and, for action nodes, d_i (Algorithm 4). This represents the effect of the state on the return status (and decision) of L_i . After adding edges for each leaf node, we increment the values in τ for the variables in $\mathcal{X}_i$ and their causal ancestors.

In the second step, which is exclusive to action nodes, we add the latest version (defined by τ) of every output variable $X \in \mathcal{Y}_i$ to $\mathcal{V}$, and we relate the execution E_i of L_i (and the decision d_i for action nodes) to their values (Algorithm 5). This represents the effect executing L_i (and the particular decision taken by the node) has on the state.

As the counts in τ are incremented after adding incoming edges to a leaf node, we can ensure that different temporal stages of the state are represented with different variables, and thus the explanation graph remains acyclic. For example, if L_i both reads from and writes to X (i.e. $X \in \mathcal{X}_i \cap \mathcal{Y}_i$), then this update is reflected in the edges $X^{(t)} \rightarrow d_i$ and $d_i \rightarrow X^{(t+1)}$, rather than a cycle.

Finally, after all leaf nodes have had their input and output edges added, we relate each top-level variable (i.e. variables without parents in $\mathcal{G}_S$) to themselves temporally, so that a variable can retain its previous value if a leaf node does not alter it (Algorithm 8 in Appendix A). We perform this “temporal linking” only for nodes without parents, as we assume other state variables have their values determined entirely by their parents. If such a variable also relies on its own previous values, these links can also be added.

5.2.3. Explanation Model Functions

Having defined the explanation graph $\mathcal{G}$ in the previous sections, all that remains to define the explanation model M is to define the ranges $\mathcal{R}$ of values and the functions $\mathcal{F}_n(\mathbf{v})$, where $\mathbf{v}$ is an assignment of variables in $\mathcal{V}$, for each node n in the causal model. The definitions of these functions are described in this section, and depicted in pseudocode form in Algorithm 9 in Appendix A.

Algorithm 4 Inserting nodes and edges based on $\mathcal{X}_i$

```

1: function ADDLEAFINPUTS( $\mathcal{G}, L_i, \tau$ )
2:   for  $X \in \mathcal{X}_i$  do
3:     Add  $X^{(\tau(X))}$  to  $\mathcal{V}$  if  $X^{(\tau(X))} \notin \mathcal{V}$ 
4:     Let  $C_X$  be the set of ancestors of  $X$  in  $\mathcal{G}_S$ 
5:      $\forall X_C \in C_X$ , add  $X_C^{(\tau(X_C))}$  to  $\mathcal{V}$  if  $X_C^{(\tau(X_C))} \notin \mathcal{V}$ 
6:     for  $X_a \rightarrow X_b \in \mathcal{E}_S$  do
7:       if  $X_a$  or  $X_b \in C(X) \cup \{X\}$  then
8:         Add edge  $X_a^{(\tau(X_a))} \rightarrow X_b^{(\tau(X_b))}$  to  $\mathcal{E}$ 
9:     Add edge  $X^{(\tau(X))} \rightarrow r_i$  to  $\mathcal{E}$ 
10:    if  $L_i$  is an action node then
11:      Add edge  $X^{(\tau(X))} \rightarrow d_i$  to  $\mathcal{E}$ 
12:    Let  $C$  be the set of ancestors of all  $X \in \mathcal{X}_i$  in  $\mathcal{G}_S$ 
13:    for  $X \in \mathcal{X}_i \cup C$  do
14:       $\tau(X) \leftarrow \tau(X) + 1$ 
15:    return  $\mathcal{G}, \tau$ 

```

Algorithm 5 Inserting nodes and edges based on $\mathcal{Y}_i$

```

1: function ADDLEAFOUTPUTS( $\mathcal{G}, L_i, \tau$ )
2:   for  $X \in \mathcal{Y}_i$  do
3:     Add  $X^{(\tau(X))}$  to  $\mathcal{V}$  if  $X^{(\tau(X))} \notin \mathcal{V}$ 
4:     Add edge  $E_i \rightarrow X^{(\tau(X))}$  to  $\mathcal{E}$ 
5:     Add edge  $d_i \rightarrow X^{(\tau(X))}$  to  $\mathcal{E}$ 
6:   return  $\mathcal{G}$ 

```

Recall that each node in the explanation graph represents either a state variable $X^{(t)}$ or the return status (r_i), execution (E_i) or decision (d_i) of a BT node $\mathcal{T}_i$. For execution variables E_i , $\mathcal{R}(E_i) = \{\text{true}, \text{false}\}$ by definition. The value of $\mathcal{F}_{E_i}$ depends on the relationship of $\mathcal{T}_i$ to its parent and its siblings (Algorithm 10, Appendix A). If $\mathcal{T}_i$ is a root node, then E_i is true, as the root node of the BT is always executed. Otherwise, $\mathcal{T}_i$ must be the child of a composite node $\mathcal{T}_p = Pa(\mathcal{T}_i)$. If $\mathcal{T}_i$ is the left-most child of $Pa(\mathcal{T}_i)$, then it is always executed first and thus $E_i = E_p$. Otherwise, the value of E_i depends on the return status of the sibling of $\mathcal{T}_i$ immediately to its left, $\mathcal{T}_j$. If $\mathcal{T}_p$ is a sequence node, then $\mathcal{T}_i$ is only executed if $\mathcal{T}_j$ succeeds, and thus E_i is true only if $r_j = \text{Success}$. If instead $\mathcal{T}_p$ is a fallback node, then $\mathcal{T}_i$ is only executed if $\mathcal{T}_j$ fails, and thus E_i is true only if $r_j = \text{Failure}$.

For return status variables r_i , $\mathcal{R}(r_i) = \{\text{Running}, \text{Success}, \text{Failure}, \text{Invalid}\}$ by definition (although *Running* can be excluded from $\mathcal{R}(r_i)$ in the case of condition nodes). The function $\mathcal{F}_{r_i}$ differs depending on the type of node of $\mathcal{T}_i$ (Algorithm 11, Appendix A). In all cases, $r_i = \text{Invalid}$ if E_i is false, by definition. Otherwise, if $\mathcal{T}_i$ is a leaf node, then r_i depends on the state $\mathbf{s}$ at the time of node execution, and thus the value of the node is $r_i(\mathbf{s})$ ($r_i(d_i, \mathbf{s})$ for action nodes). If $\mathcal{T}_i$ is a composite node, its return status depends on the return statuses of its children. For a sequence node, $r_i = \text{Success}$ if all of its children return *Success*, otherwise taking the value of the first dissenting child. Similarly for a fallback node, $r_i = \text{Failure}$ if all of its children return *Failure*, otherwise taking the value of the first dissenting child.

For decision variables d_i , the range can be determined by the node's action space, $\mathcal{R}(d_i) = \mathcal{A}_i$. The value of $\mathcal{F}_{d_i}$ can easily be determined as either $\emptyset$ if E_i is false, by definition, or else by evaluating $d_i(\mathbf{s})$ (Algorithm 12, Appendix A).

Finally, for state variables $X^{(t)} \hat{=} X \in \mathcal{X}$, the range is already specified by the state model M_S , such that $\mathcal{R}(X^{(t)}) = \mathcal{R}_S(X)$. To determine $\mathcal{F}_{X^{(t)}}$, we note the distinction between internal state variables, whose values can be set by the BT, and external state variables, whose values cannot be set by the BT. The value of an external state variable, whose parents in $\mathcal{G}$ are all state variables, is determined by the corresponding function $\mathcal{F}_{SX}$ from the state causal model (Algorithm 13, lines 1-3, Appendix A). For internal state variables, the value is instead determined by p_i for the leaf L_i where E_i and d_i are parents of $X^{(t)}$ in $\mathcal{G}$ (Algorithm 13, lines 4-13, Appendix A), unless L_i was not executed, in which case the value remains the same (i.e. $X^{(t)} = X^{(t-1)}$).

5.3. Explanation Generation

Given the episodic memory of a particular execution (Section 5.1) and the explanation model derived from the structure of a BT and domain knowledge of the state and decision-making (Section 5.2), we can now query the value of any node in the explanation model in order to generate explanations, automatically and in real time, about the BT execution.

Recall that a query takes the form $why(A, B)$, where A and B are mutually exclusive statements (see Section 4.2). When querying the explanation model, we set $A = \{V = v \text{ at time } k | V \in \mathcal{V}_Q\}$, where $\mathcal{V}_Q \subseteq \mathcal{V}$ denotes the set of variables we are querying and $V = v$ at time k must be true (i.e. v is the value of V at time k in the episodic memory). Similarly, we set $B = \{V \in \mathcal{R}_Q(V) | V \in \mathcal{V}_Q\}$, where $\mathcal{R}_Q(V) \subset \mathcal{R}(V)$ (with $v \notin \mathcal{R}_Q(V)$) denotes a set of untrue (i.e. foil) values of V .

For example, consider the example query presented in Section 4.2. Supposing that there exists a node in the BT *RepeatOrEnd* responsible for deciding whether to repeat a sequence or end the task, we can formally rephrase the query as $why(d_{RepeatOrEnd} = End \text{ at time } k, d_{RepeatOrEnd} = Repeat \text{ at time } k)$, where k is the time step at which the *RepeatOrEnd* node executed.

In order to generate an explanation for such a query, we must first recover the state of the BT and state variables at the queried time (k). This requires first initialising the explanation model with the initial values stored in the episodic memory and then iterating through the episodic memory from the initial time (0) until time k , updating values as updates occur. At the end of each “tick” prior to time k , the state of the tree can be reset and the values of the latest temporal versions of each state variable can be transferred to their initial corresponding versions. The result is that the true values of all variables in $\mathcal{V}_Q$ (and their ancestors in $\mathcal{G}$) are known at the queried time k .

At this point, a set of explanations can be generated by performing a counterfactual search (Algorithm 6, visualised in Figure 9, Appendix A). This search is driven by testing interventions (performed via the *Do* operator) on the ancestors of the variables in $\mathcal{V}_Q$ to see if they elicit any changes in the queried variables that would satisfy the query, following similar approaches by Albini, Rago, Baroni, Toni et al. (2020) and Diehl and Ramirez-Amaro (2022). Intuitively, if setting $X_n^{(t)} = x'_n$ instead of x_n results in a change in the variables in $\mathcal{V}_Q$ such that the query is satisfied, then $X_n^{(t)} = x_n$ is a reason for the true values of the variables in $\mathcal{V}_Q$.

More formally, the search operates by first reducing the search space to only consider the ancestors of the variables in $\mathcal{V}_Q$. For every variable V in the reduced variable set $\mathcal{V}^*$, we add every possible setting $V = v^*$ (except $V = v$, the true value of the variable) to a list of potential explanations (Algorithm 6, lines 25 - 31). Discretisation of continuous variables is employed to make the search tractable. One by one, we can perform the intervention $Do(V = v^*)$ on the causal model and check if the new values of $V_Q \in \mathcal{V}_Q$ satisfy the query (i.e. if $V = v'$ after intervention, then if $v' \in \mathcal{R}_Q(V) \forall V \in \mathcal{V}_Q$, the query is satisfied).

If, after intervening $Do(V = v^*)$, the query is satisfied, then we can construct an explanation around V with the reason $\mathbf{R} = \{V = v\}$ and counterfactual $\mathbf{C} = \{(\{V = v^*\}, \{V_Q = v' | V_Q \in \mathcal{V}_Q\})\}$. We can read this explanation as “ $V_Q = v_Q \forall V_Q \in \mathcal{V}_Q$ because $V = v$. If instead $V = v^*$, then $V_Q = v' \forall V_Q \in \mathcal{V}_Q$.”

If no explanations using only a single explanation variable are found, we can perform a “deeper” search by considering interventions on two or more variables simultaneously, until either an explanation is found or the search has reached a maximum depth. At the end of the process, the algorithm will return all explanations consisting of i variables, where i is the minimum depth for which explanations are found.

With such a search, we can return a set of explanations for any query regarding the execution E_i , return status r_i or decision d_i of a BT node, or the value of a state variable $X^{(t)}$, the accuracy of which is limited only by the completeness of the explanation model, which is determined by the domain knowledge used to construct it, and the granularity of the discretisation in the case of continuous variables.

Once explanations been provided, a user could make further “follow-up” queries based on the explanations received. These queries can be used to clarify an explanation, or dig deeper towards a root cause (e.g. if the user asks $why(A, B)$ and receives an explanation $\langle \{X = x\}, \{(X = x', B)\} \rangle$, a natural follow-up question could be $why(X = x, X = x')$. Similarly, follow-up questions allow a user to extend the time window of explanations beyond a single BT “tick”, by recognising that the value of $X_i^{(0)}$ at tick t is the same as $X_i^{(\tau(X_i))}$ at tick $t - 1$. Thus for explanation $\langle \{X_i^{(0)} = x\}, \{(X_i^{(0)} = x', B)\} \rangle$, the follow-up can be $why(X_i^{(\tau(X_i))} = x, X_i^{(\tau(X_i))} = x')$ in the context of the previous tick.

Algorithm 6 Counterfactual search given query Q

```

1: function COUNTERFACTUALSEARCH( $Q, D_{max}$ )
2:    $\mathcal{V}^* \leftarrow \mathcal{V}_Q \cup \{V \mid V \text{ is an ancestor of } W \in \mathcal{V}_Q \text{ in } \mathcal{G}\}$ 
3:    $\mathcal{E}^* \leftarrow \{v \rightarrow w \mid v, w \in \mathcal{V}^*, v \rightarrow w \in \mathcal{E}\}$ 
4:    $\mathcal{G}^* \leftarrow (\mathcal{V}^*, \mathcal{E}^*)$ 
5:    $\mathcal{M}^* \leftarrow (\mathcal{G}^*, \mathcal{R}, \mathcal{F})$ 
6:    $\mathbf{E} \leftarrow \emptyset, i \leftarrow 0$ 
7:   while  $\mathbf{E} = \emptyset$  and  $i \leq D_{max}$  do
8:      $\mathbf{E}^* \leftarrow \text{EXPLAINTODEPTH}(Q, \mathcal{M}^*, i)$ 
9:     Add  $\mathbf{E}^*$  to  $\mathbf{E}$ 
10:     $i \leftarrow i + 1$ 
11:   return  $\mathbf{E}$ 
12: function EXPLAINTODEPTH( $Q, \mathcal{M}^*, i$ )
13:    $\mathbf{E} \leftarrow \emptyset$ 
14:    $\mathcal{V}_{Search} \leftarrow \mathcal{V}^* \setminus \mathcal{V}_Q$ 
15:    $C \leftarrow \text{COMBINATIONS}(\mathcal{V}_{Search}, i)$ 
16:   for assignment  $c \in C$  do
17:      $\mathcal{G}' \leftarrow Do(\mathcal{M}^*, V = v^* \forall (V, v^*) \in c)$ 
18:     Let  $v'$  denote the value of  $V$  in  $\mathcal{G}'$ 
19:     if  $v' \in \mathcal{R}_Q(V) \forall V \in \mathcal{V}_Q$  then
20:        $\mathbf{R} \leftarrow \{V = v \mid (V, v^*) \in c\}$ 
21:        $\mathbf{J} \leftarrow \{V = v^* \mid (V, v^*) \in c\}$ 
22:        $\mathbf{K} \leftarrow \{V = v' \mid V \in \mathcal{V}_Q\}$ 
23:       Add  $\langle \mathbf{R}, (\mathbf{J}, \mathbf{K}) \rangle$  to  $\mathbf{E}$ 
24:   return  $\mathbf{E}$ 
25: function COMBINATIONS( $\mathcal{V}_{Search}, i$ )
26:   combinations  $\leftarrow \emptyset$ 
27:   for each subset  $\mathcal{V}_i$  of  $\mathcal{V}_{Search}$  of size  $i$  do
28:     for each assignment  $V = v^* \forall V \in \mathcal{V}_i$  do
29:       if  $v^* \neq v \forall V \in \mathcal{V}_i$  in the assignment then
30:         Add assignment to combinations
31:   return combinations

```

5.4. Properties

Having presented the entire approach to building an explanation model of the BT and state and using it to generate counterfactual explanations, we now discuss some properties of this approach.

Guidotti (2022) identifies a number of desirable properties for counterfactual explanations, several of which achieved through the design of our approach. For example, a counterfactual explanation is considered *valid* if the counterfactual situation proposed does indeed result in the foil decision. All explanations returned by Algorithm 6 are *valid* by construction, as only explanations that satisfy the query are included in the explanation set (Algorithm 6, line 19). Similarly, our explanations are *minimal* - meaning the number of variable value changes between the real assignment and the counterfactual is minimal - as the search incrementally increases the number of variables changed (the “depth”) until at least one explanation is found (Algorithm 6, line 7).

Likewise, the explanation set is maximally *diverse* - meaning the returned counterfactuals are as different from each other as possible - as the search covers all combinations of a given depth and thus changes to all potential variable combinations are considered (Algorithm 6, lines 25-31). The explanations also respect the *causal* relationships in the BT and the environment, as this information is provided by the structure of the BT and the domain knowledge passed to the system. Ultimately, the causal fidelity is limited by the fidelity of the domain knowledge, as discussed in Section 6.3.

Other properties such as *plausibility*, *actionability* and *discriminative power* identified by Guidotti (2022) are not accounted for by our approach, although they could easily be incorporated by further restricting the search space of explanations. For example, *discriminative power* could be improved by only considering variables for which all

changes result in a valid counterfactual outcome, as discussed by Albini et al. (2020). *Plausibility* could be improved by encoding some notion of which state configurations are more probable than others. It is worth noting that the property of causality already overlaps somewhat with plausibility, and that any interventions on the explanation model in our approach will have plausible downstream effects, even if the intervention itself may be implausible.

5.4.1. Complexity of the Explanation Model

Based on the node insertions in Algorithms 2 and 3, we note that the number of nodes in the explanation model grows with the number of nodes in the BT (especially leaf nodes), the number of variables in the state model ($|\mathcal{X}_S|$) and the sizes of the input and output sets ($|\mathcal{X}_I|$ and $|\mathcal{Y}_I|$), as well as the number of causal ancestors of the input variables in the state model.

More formally, let $\mathcal{T}$ denote the set of BT nodes, with $\mathcal{L} \subset \mathcal{T}$ denoting the set of leaf nodes and $A \subseteq \mathcal{L}$ denoting the set of action nodes. Assume a worst-case scenario in which every state variable is both read and potentially written to by every BT node. In other words, we assume that $\mathcal{X}_i = \mathcal{X}_S \forall L_i \in \mathcal{L}$ and that $\mathcal{Y}_i = \mathcal{X}_S \forall T_i \in \mathcal{T}$. Given that Algorithm 2 always inserts $2|\mathcal{T}| + |A|$ nodes in the explanation model, and Algorithm 3 inserts a node into the model every time a state variable appears in $\mathcal{X}_i$ (or an ancestor of such a variable in $\mathcal{G}_S$) or in $\mathcal{Y}_i$, then the number of nodes in the explanation model grows asymptotically with $\mathcal{O}(2|\mathcal{T}| + |A| + |\mathcal{X}_S|(|\mathcal{L}| + |A|))$ which simplifies to $\mathcal{O}(|\mathcal{T}| + |\mathcal{X}_S| \times |\mathcal{L}|)$, and is in turn bounded above by the simpler $\mathcal{O}(|\mathcal{X}_S| \times |\mathcal{T}|)$.

To determine how the number of edges in the explanation model scales with BT and state model size, we once again assume the worst-case scenario of $\mathcal{X}_i = \mathcal{Y}_i = \mathcal{X}_S$, and also assume that the state model $\mathcal{M}_S$ contains the maximum number of edges, which for a directed acyclic graph is $\frac{1}{2}|\mathcal{X}_S|(|\mathcal{X}_S| - 1)$. From Algorithm 2, we observe that two edges are added for each node (except the root of the tree, for which only one is added), and a further two edges are added for each action node. Additionally, one edge is added for each parent-child relationship in the tree, which is always exactly $|\mathcal{T}| - 1$ edges due to the properties of trees. From Algorithm 3, we observe that for each leaf node, edges are added for every variable in $\mathcal{X}_i$, replicating the edges in $\mathcal{E}_S$ for each temporal version of the state variables. Edges are also added for each action node proportional to the number of input and output variables. Edges linking temporal versions of top-level state variables (of which there is only one in a maximally connected DAG) are added for each action node potentially changing that variable's value. Taken altogether, the number of edges in the explanation model grows asymptotically with $\mathcal{O}(3|\mathcal{T}| + 2|A| + 0.5|\mathcal{X}_S|^2 \times |\mathcal{L}| + 0.5|\mathcal{X}_S| \times |\mathcal{L}| + |A| \times |\mathcal{X}_S|)$, which simplifies to $\mathcal{O}(|\mathcal{X}_S|^2 \times |\mathcal{L}| + |\mathcal{T}|)$, and is in turn bounded above by the simpler $\mathcal{O}(|\mathcal{X}_S|^2 \times |\mathcal{T}|)$.

These analyses highlight the important role the size of the BT and the state model have in determining the size of the explanation model, which ultimately impacts the runtime complexity of the counterfactual search (as we expand on in Section 5.4.2).

5.4.2. Complexity of the Counterfactual Search

We now examine the runtime complexity of the counterfactual search algorithm (Algorithm 6) for an explanation model with N nodes. We begin by observing that the expensive operations that occur during the execution of the search are the application of the *Do* operator on the reduced sub-model $\mathcal{M}^*$, as well as checking to see if the intervention has satisfied the query Q . Both operations occur once for each combination of variable assignments (the set of which is labelled C) considered and for each level of depth considered up to the maximum depth D_{max} .

Applying the operation $Do(X = x)$ deletes the incoming edges to X , changes the value to x and then propagates the effects of the change to every descendent of X in $\mathcal{M}^*$. First of all, let us assume the worst-case scenario in which the queried variable has as many ancestors as possible. To create this upper bound, we assume $\mathcal{M}^* = \mathcal{M}$, and thus every node in the explanation model must be considered in the counterfactual search. We also assume a worst-case cost of $\mathcal{O}(F)$ for each function in $\mathcal{F}$. In the absolute worst case, a given $Do(X = x)$ necessitates $N - 1$ function executions, and thus runtime complexity for the *Do* operator in this scenario is $\mathcal{O}(NF)$.

The search considers multiple interventions on a single variable, one for each possible value the variable can take (discretised over a range, in continuous cases). In a worst-case simplification, we assume each node in the explanation model has R_{max} possible interventions that must be considered. Therefore, at a given depth D , there are $\binom{N}{D} R_{max}^D$ combinations of assignments, and thus, taken together with the runtime complexity of the *Do* operator, we have a runtime complexity of $\mathcal{O}(NF \binom{N}{D_{max}} R_{max}^{D_{max}})$. Given that N grows asymptotically bounded above by $\mathcal{O}(|\mathcal{X}_S| \times |\mathcal{T}|)$, we can express the counterfactual search runtime complexity in terms of BT and state model size as $\mathcal{O}(|\mathcal{X}_S| \times |\mathcal{T}|) F \binom{|\mathcal{X}_S| \times |\mathcal{T}|}{D_{max}} R_{max}^{D_{max}}$.

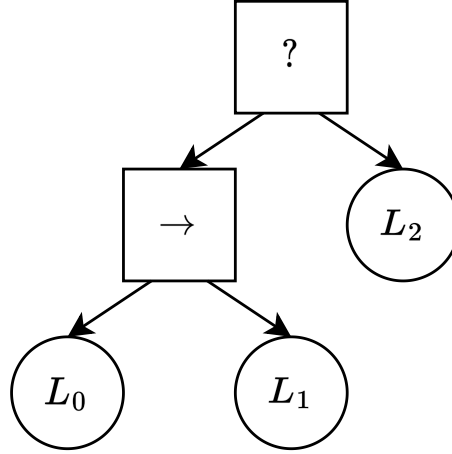


Figure 6: An example BT with a fallback node $\mathcal{T}_?$, sequence node $\mathcal{T}_\rightarrow$, condition node L_0 and action nodes L_1 and L_2 . The leaf nodes in this tree have the following input, output and action sets: $\mathcal{X}_0 = \{X_a\}$, $\mathcal{X}_1 = \{X_a, X_c\}$, $\mathcal{X}_2 = \{X_d\}$, $\mathcal{Y}_0 = \emptyset$, $\mathcal{Y}_1 = \{X_b\} = \mathcal{Y}_2$, $\mathcal{A}_0 = \emptyset$, $\mathcal{A}_1 = \{a_0, a_1\}$, $\mathcal{A}_2 = \{a_2, a_3\}$

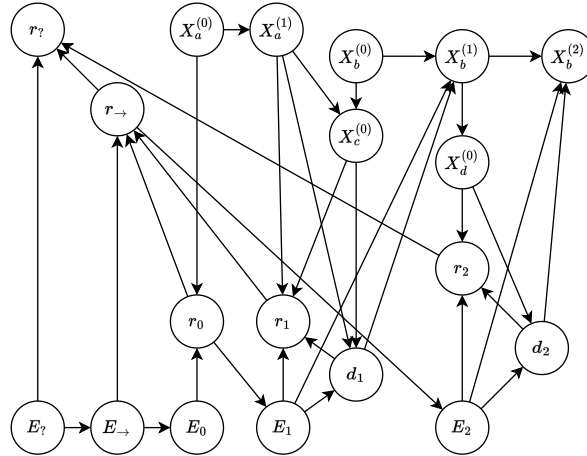


Figure 7: The complete explanation model for the behaviour tree in Figure 6 and the state graph in Figure 5, constructed using Algorithm 1.

It is worth recalling that the runtime complexity calculated above assumes the worst-case scenario, including for the state and explanation model structures, possible variable values and BT node input/output sets. In particular while a high value of D_{max} may be set, early stopping at lower depths greatly reduces runtime. Additionally, the complexity $\mathcal{O}(F)$ is often $\mathcal{O}(1)$ in typical BT implementations (for example, in nodes where preconditions are checked or flags are set). The empirical analysis in Section 7 shows runtime to be low, even for the larger and more complex BT examined in Section 7.2.

6. Case Study

To demonstrate how our architecture is able to generate explanations for a wide range of queries, we provide some illustrative examples using a simple toy behaviour tree (depicted in Figure 6) and state model (in this case, identical to the example in Figure 5, with the assumption that each variable is a boolean statement).

Regardless of the query, we first need to construct an explanation model using the algorithms presented in Section 5.2. In this case, the resulting causal model is depicted in Figure 7. Using this model, we can query the return statuses, execution and decisions of BT nodes as well as the values of state variables.

Natural Language Query	Formal Query	Explanation Set	Natural Language Explanation
"Why did L_0 execute?"	$why(E_0 = True \text{ at time } 1, E_0 = False)$	$e_1 = \langle E_{\rightarrow} = True, (E_{\rightarrow} = False, E_0 = False) \rangle$ $e_2 = \langle E_{\rightarrow} = True, (E_{\rightarrow} = False, E_0 = False) \rangle$	"...because $\mathcal{T}_{\rightarrow}$ executed..." "...because $\mathcal{T}_{\rightarrow}$ executed..."
"Why didn't L_1 execute?"	$why(E_1 = False \text{ at time } 2, E_1 = True)$	$e_1 = \langle r_0 = Failure, (r_0 = Success, E_1 = True) \rangle$ $e_2 = \langle X_a^{(0)} = False, (X_a^{(0)} = True, E_1 = True) \rangle$	"...because $\mathcal{T}_0$ failed..." "...because $X_a^{(0)}$ was false..."
Why did L_2 choose a_2 instead of a_3 ?	$why(d_2 = a_2 \text{ at time } 3, d_2 = a_3)$	$e_1 = \langle X_b^{(0)} = False, (X_b^{(0)} = True, d_2 = a_3) \rangle$ $e_2 = \langle X_b^{(1)} = False, (X_b^{(1)} = True, d_2 = a_3) \rangle$ $e_3 = \langle X_d^{(0)} = True, (X_d^{(0)} = False, d_2 = a_3) \rangle$	"...because $X_b^{(0)}$ was false..." "...because $X_b^{(1)}$ was false..." "...because $X_d^{(0)}$ was true..."
"Why was X_c false?"	$why(X_c = False \text{ at time } 2, X_c = True)$	$e_1 = \langle X_a^{(1)} = False \cap X_b^{(0)} = False, (X_a^{(1)} = True \cap X_b^{(0)} = True, X_c = True) \rangle$	"...because $X_a^{(1)}$ was false and $X_b^{(0)}$ was false..."

Table 1

Some example queries and their corresponding explanations, given the explanation model in Figure 7. For this case study, we have L_2 select a_2 if $X_d = True$, otherwise selecting a_3 . We also have $X_c = X_a \cap X_b$ and $X_d = \neg X_b$. Note that the natural language queries and explanations (only reasons, for the sake of brevity) are included purely for the purposes of legibility. Our architecture takes in a formal query as input and produces the formal explanation set as output.

6.1. Step-by-Step Example

For example, suppose that the condition node L_0 has implemented in such a way as to return *Success* if X_a is true and *Failure* otherwise. If we were to set $X_a = False$ and execute the BT, finding that $r_0 = Failure$ at time 1, a valid query may be to ask $why(r_0 = Failure \text{ at time } 1, r_0 = Success)$. We now provide a step-by-step description of Algorithm 6 for this particular query to illustrate how it arrives at an explanation.

Firstly, the algorithm identifies a subgraph $\mathcal{G}^*$ containing only the query variables $\mathcal{V}_Q$ and their ancestors in $\mathcal{G}$. Examining the explanation model in Figure 7, we arrive at the set $\mathcal{V}^* = \{E_{\rightarrow}, E_{\rightarrow}, E_0, X_a^{(0)}, r_0\}$ for query variable $\mathcal{V}_Q = \{r_0\}$. The next step is to identify all possible interventions in the search space $\mathcal{V}^* \setminus \mathcal{V}_Q$, which, given that each variable in the set happens to be a boolean statement, is $C = \{E_{\rightarrow} = False, E_{\rightarrow} = False, E_0 = False, X_a^{(0)} = True\}$. Note how the true values of these variables (e.g. $E_0 = True$ or $X_a^{(0)} = False$) are omitted from the search space.

For each statement $V = v^*$ in C , we can perform the intervention $Do(V = v^*)$ to see if the resulting values in $\mathcal{M}^*$ satisfy our query. In this case, to satisfy the query, the value of r_0 after the intervention must be *Success*. We are able to obtain the counterfactual value of r_0 as we are able to run the node with new input variables, thus evaluating the function $r_0(\mathbf{s})$. Attempting each intervention, we note that $Do(E_{\rightarrow} = False) \implies r_0 = Invalid$, $Do(E_{\rightarrow} = False) \implies r_0 = Invalid$, $Do(E_0 = False) \implies r_0 = Invalid$ and $Do(X_a^{(0)} = True) \implies r_0 = Success$. As only the intervention $Do(X_a^{(0)} = True)$ satisfies the query, we arrive at a single explanation $\langle \mathbf{R} = \{X_a^{(0)} = False\}, \mathbf{C} = (\{X_a^{(0)} = True\}, \{r_0 = Success\}) \rangle$, which in natural language could be converted to " L_0 failed because $X_a^{(0)}$ was false. If $X_a^{(0)}$ was true, L_0 would have succeeded.". Of course, this is a simple example as r_0 has only a few ancestors in the causal model. The query $why(r_{\rightarrow} = Failure \text{ at time } t, r_{\rightarrow} = Success)$, for example, would result in 8 possible explanations due to the much larger set of ancestors of $r_{\rightarrow}$.

The same algorithm can be applied to query any variable in the explanation model. Example queries and corresponding explanations are given in Table 1. In particular, note the explanations provided for the first example, $why(E_0 = True \text{ at time } 1, E_0 = False)$. These explanations note that the execution L_0 is the result of the chain of executions of its ancestors, $\mathcal{T}_{\rightarrow}$ and $\mathcal{T}_{\rightarrow}$.

6.2. Comparison with Other Behaviour Tree Methods

Using the BT depicted in Figure 6, we now compare our approach to that of Han et al. (2021) to illustrate how our approach extends both the types of causal queries that can be provided to the system as well as the types of reasons that can be included in the explanation. In addition to other types of queries which are not within the scope of our work, the method of Han et al. (2021) is capable of answering the question "Why are you doing this?". Within our framework of contrastive queries, this question can be expressed as the formal query " $why(E_i = True, E_i = False)$ ", where E_i is the currently executing node. For this example, suppose the node of interest is L_1 and that we set $X_a = True$, such that L_0 returns *Success* and thus $E_i = E_1 = True$. To answer the query, the method of Han et al. (2021) would return a non-decorator ancestor of L_1 , in this case $\mathcal{T}_{\rightarrow}$ (assuming the two nodes have different names). Interpreting this causally, we would have the reason $E_{\rightarrow} = True$. Our method, on the other hand, would also include $E_{\rightarrow} = True$,

$E_0 = \text{True}$, $r_0 = \text{Success}$ and $X_a^{(0)} = \text{True}$ as reasons, and would provide counterfactuals for each, demonstrating that our approach allows for more types of reasons to be considered in the counterfactual search.

It is worth noting that the method of Ögren and Alfredson (2023) would also include these reasons in the explanation, assuming that the BT in Figure 6 is transformed to a suitable “Make-sure” BT. However, both this method and that of Han et al. (2021) are limited in the types of causal queries which the system can respond to. As demonstrated in Table 1, our method can answer queries about decisions (d_i), return statuses (r_i) and state variable values in addition to execution statuses (E_i). Our method can answer queries about events which did not occur (e.g. “ $\text{why}(E_1 = \text{False}, E_1 = \text{True})$ ”), and can consider specific contrastive foils when generating explanations (e.g. “ $\text{why}(d_2 = a_2, d_2 = a_3)$ ”).

6.3. The Importance of Domain Knowledge

Thus far we have considered explanations generated from the complete explanation model, built from the structure of the BT (Section 5.2.1) as well as domain knowledge in the form of a state model and input/output/action sets for each BT node (Section 5.2.2). While the BT structure is always known, the domain knowledge used to construct the explanation model may be incomplete or incorrect (e.g. due to model simplifications or misspecifications). We now examine the effect of missing or additional edges in the explanation model to highlight the importance of this domain knowledge.

Firstly, consider that, for a given leaf node L_i , edges between nodes representing temporal versions of state variables and r_i , d_i and E_i are determined by the sets $\mathcal{X}_i$, $\mathcal{Y}_i$ and $\mathcal{A}_i$, representing the inputs, outputs and action space of the node respectively. If these sets were not known, we would not be able to effectively link changes to the state to the decisions and outcomes of BT nodes (and vice-versa). In order to be able to include these relationships, we would have to add the edges $X^{(t)} \rightarrow r_i$, $X^{(t)} \rightarrow d_i$, $d_i \rightarrow X^{(t+1)}$ and $E_i \rightarrow X^{(t+1)} \forall X \in \mathcal{X}$ to the graph, greatly increasing the number of edges and thus the size of the search space when performing the counterfactual search. Thus, the inclusion of $\mathcal{X}_i$, $\mathcal{Y}_i$ and $\mathcal{A}_i$ serve to reduce the size and complexity of the counterfactual search.

If instead these sets are present but are incorrect, such that elements that should be contained in these sets are excluded, then certain explanations may not be findable by the counterfactual search. For example, consider the explanation model in Figure 7. If we were to set $\mathcal{Y}_1 = \emptyset$ rather than $\mathcal{Y}_1 = \{X_b\}$, then the edges $d_1 \rightarrow X_b^{(1)}$ and $E_1 \rightarrow X_b^{(1)}$ would be excluded from the graph. Therefore, any interventions made to r_1 , d_1 , and their ancestors (which are not also ancestors to $X_b^{(1)}$) would not be considered in the counterfactual search and thus would not appear in explanations querying $X_b^{(1)}$ and its descendents. This highlights a trade-off between the search efficiency introduced by including the decision-making knowledge and the possibility of missing explanations due to the inclusion of misspecified input, output and action sets.

Secondly, consider that all edges $X_m^{(t_i)} \rightarrow X_n^{(t_j)}$ where $m \neq n$ are derived from knowledge introduced by the state model $\mathcal{M}_S$. If the state model is missing or incomplete, then certain explanations would be excluded from the counterfactual search results. For example, returning to the model in Figure 7, if the edge $X_b \rightarrow X_d$ were to be excluded, then the edge $X_b^{(1)} \rightarrow X_d^{(0)}$ would not be present in the explanation model. Thus, while $X_d^{(0)}$ would potentially appear in explanations querying r_2 , $X_b^{(1)}$ would not, and neither would its ancestors, such as E_1 and d_1 .

In summary, the accuracy and completeness of the explanations generated by a counterfactual search over the model are limited by the accuracy and completeness of the model (as is true for any approach which relies on causal models), which are in turn determined by the fidelity of the domain knowledge used to construct the model.

7. Evaluation

Having provided simple illustrative examples to demonstrate the effectiveness of the algorithms proposed in Section 5, we now focus on a more systematic evaluation of the explanations generated by our architecture. There are many different approaches to evaluating explanations. Doshi-Velez and Kim (2017) identify three broad types of evaluation depending on the evaluation context and the presence of human evaluators. *Application-* and *human-grounded* approaches both involve user studies, the primary difference being the fidelity to the intended application domain. On the other hand, *functionally-grounded* approaches define an appropriate proxy measure which can be evaluated automatically. Evaluations both with and without user studies can be insightful, depending on the particular property of the explanations being measured (Nauta, Trienes, Pathak, Nguyen, Peters, Schmitt, Schlötterer, Van Keulen and Seifert, 2023). Given that explanation generation is only one component of the broader explainability interaction (which also includes converting a human’s request into a formal explanation query and communicating a formal

explanation in an effective manner (Anjomshoe, Najjar, Calvaresi and Fr  mling, 2019; Matarese, Rea and Sciutti, 2021)), we opt for a functionally-grounded approach in order to validate our architecture in isolation.

Several desirable properties have been identified both for explanations in general (Nauta et al., 2023) and counterfactual explanations in particular (Guidotti, 2022), as discussed in Section 5.4. We focus our evaluations on measuring the central property of *correctness* (otherwise known as *fidelity*), which describes the faithfulness of the explanations to the model being explained. Nauta et al. (2023) identify several metrics for evaluating correctness, including the *controlled synthetic data check* - which operates by creating a synthetic dataset such that the model under evaluation reasons in a certain way and comparing the explanations against this ground-truth reasoning - and the *single/incremental deletion* - which operates by perturbing or removing one or more features and comparing the size of the effect on the model to the importance score given by a feature importance explanation. Such approaches are aimed at evaluating explanations of black box models learned from data, the absence of which limits their applicability to our architecture.

To measure correctness and demonstrate that our counterfactual search reflects the reasoning of the BT and the dynamics of the environment (as captured by the state model), we therefore introduce the *target recovery* metric, inspired other evaluation approaches that perturb the initial system input, such as the *controlled synthetic data check* identified by Nauta et al. (2023). Here we outline the procedure for calculating the metric.

We begin by labelling an arbitrary initial state configuration as the “default” initialisation, resulting in a corresponding “default behaviour” by the BT. We note the particular sequence of return statuses, executions and actions produced by the default behaviour. We then perturb the initial state by altering the value of a single state variable, resulting in an “altered behaviour” by the BT. We denote this perturbed variable as the “target variable”. If the default and altered behaviours consist of an identical sequence of node return statuses, executions and decisions (as might occur if a small change was made, for example), then we cannot apply the evaluation. If, however, there is a deviation between the altered behaviour and the default behaviour, we can construct a query from that difference. For example, suppose that the default behaviour results in the event $A = a$ at time t , and that the altered behaviour is identical to the default behaviour for times $\{0, \dots, t - 1\}$ but has $A = a'$ at time t . We can therefore pose the query *why*($A = a'$ at time t , $A = a$) in the context of the altered behaviour and, using Algorithm 6, we can generate a set of explanations $\langle \mathbf{R}, \mathbf{C} \rangle$. We consider the *target recovery* to be successful if the target variable is contained in the reason set $\mathbf{R}$. For example, if the perturbation is $X = x'$, where the default initialisation has $X = x$, the *target recovery* is successful if $\{X = x'\} \in \mathbf{R}$.

If, over a number of different perturbations and executions, we obtain a high *target recovery* rate (*TRR*), then this suggests our algorithm is capable of accurately recovering the cause of a difference in behaviour.

All experiments described in the following sections have been executed on a device running Ubuntu 22.04.5 LTS with python 3.12.6, 62 GiB RAM, 13th Gen Intel Core i9-13900 CPU and NVIDIA GeForce RTX 4090 24 GiB GPU.

7.1. Random Domains

To demonstrate that our explanation generation method is capable of generating explanations that accurately reflect the decision-making of a wide class of BTs, we evaluate the *target recovery* metric across a number of randomly generated BTs and state dynamics.

The random BTs are determined by the number of leaf nodes (in our experiment, either 2, 4 or 8). Random tree structures are generated by incrementally adding nodes as children to existing nodes until the number of leaves matches the specified parameter and every composite node has two or more children. Composite nodes are then randomly assigned as either a sequence or fallback node, while every leaf node is randomly assigned an input, output and action set. Random execution functions are generated for each node that are coherent with these sets. The random state graphs are determined by the number of state variables (4, 8 or 12) and a connectivity parameter (0, 0.25, 0.5, 0.75, 1) which determines the number of edges. For each graph, half of the nodes are designated “top-level” nodes and have no parents. Each variable is a boolean statement, whose value is a random composition (using “and” and “or” operators) of its parents in the state graph.

For each combination of number of leaves, number of state variables and connectivity parameter, 10 BT-state graph combinations are generated using different seeds, for a total of 450 unique combinations. For each combination, a random assignment of state variable values are designated as the “default”. Additionally, we perturb each of the designated “top-level” state variables (by flipping the boolean value) to produce a number of altered executions. For each combination, we compare the altered executions with the default one to evaluate the *target recovery* metric. In total, 1800 comparisons are made. For each assignment of BT and state parameters, the *TRR* and runtime statistics are given in Table 2. Additionally, we provide the size of the explanation model (i.e. the number of nodes in the

BT/State Parameters		Number of Runs		TRR	Runtime (s)		
No. State Vars	No. BT Leaves	Target Found	No Difference		Min.	Max.	Mean (Std)
4	2	49	51	1	0.0008	0.0076	0.0026 (0.0019)
4	4	48	52	1	0.0008	0.0166	0.0032 (0.0042)
4	8	42	58	1	0.0012	0.0786	0.0170 (0.0204)
8	2	76	124	1	0.0011	0.0263	0.0053 (0.0054)
8	4	79	121	1	0.0011	0.0803	0.0089 (0.0141)
8	8	93	107	1	0.0017	0.2716	0.0355 (0.0573)
12	2	81	219	1	0.0012	0.0655	0.0132 (0.0132)
12	4	104	196	1	0.0012	0.241	0.0278 (0.0473)
12	8	115	185	1	0.0026	0.3709	0.0775 (0.0845)
All	All	687	1113	1	0.0008	0.3709	0.0266 (0.0517)

Table 2

Results for target recovery experiments with randomly generated state graphs and behaviour trees. For the sake of compactness, the connectivity parameter is omitted. In each run either the target variable is successfully recovered or there is no difference in execution. In addition to target recovery rate (TRR), we also show the runtime, which is the time taken to generate the explanation set after receiving a query.

BT/State Parameters		Number of Explanation Model Nodes			Number of Counterfactual Candidates			Number of Explanations		
No. State Vars	No. BT Leaves	Min.	Max.	Mean (Std)	Min.	Max.	Mean (Std)	Min.	Max.	Mean (Std)
4	2	10	14	11.71 (1.19)	3	14	6.51 (3.61)	1	4	2.12 (0.95)
4	4	19	29	22.52 (2.95)	3	25	6.79 (6.08)	1	5	2.10 (1.24)
4	8	33	51	41.36 (5.21)	4	52	19.88 (15.90)	1	6	2.95 (1.31)
8	2	14	24	16.83 (2.14)	3	23	8.01 (4.66)	1	7	3.04 (1.35)
8	4	22	45	29.04 (5.47)	3	43	10.67 (8.90)	1	13	3.34 (2.42)
8	8	43	77	57.73 (9.06)	5	81	22.47 (21.57)	1	13	3.98 (2.83)
12	2	18	33	23.68 (4.48)	3	31	11.86 (6.74)	1	11	3.77 (2.02)
12	4	28	60	40.82 (9.98)	3	61	17.23 (14.64)	1	12	3.83 (2.51)
12	8	50	105	77.90 (14.19)	6	88	34.04 (21.26)	1	14	5.70 (3.08)
All	All	10	105	39.96 (23.13)	3	88	17.02 (17.03)	1	14	3.71 (2.52)

Table 3

Additional results for the experiments conducted with randomly generated state graphs and behaviour trees. Here we present the number of nodes in the automatically constructed explanation model, the number of counterfactual candidates (i.e. interventions) considered during the counterfactual search, and the number of explanations in the explanation set.

causal model), the number of counterfactual candidates considered in the counterfactual search (i.e. the interventions considered when producing the explanation set) and the number of explanations provided when explaining a given altered execution in Table 3.

The majority of executions (1113, ~61.8%) result in no difference whatsoever in the sequence of node executions, return statuses and decisions made (see Table 2), which can be attributed to the particular implementation of the BT functions and the arbitrary selection of default initialisations. Nevertheless, in a large number of executions (687, ~38.2), the perturbations result in different behaviour between executions, for which a valid query regarding the difference could be automatically constructed. Across all of these executions our approach achieves 100% TRR , meaning the explanation set generated by Algorithm 6 always contains the target variable. This indicates that our algorithm is capable of producing explanations that correctly identify the cause of a difference in behaviour across a wide class of BTs and state graphs.

Examining the runtime for each assignment of BT and state parameters (recorded as the amount of time taken to generate the explanation set upon receiving a query), we see that in all cases our approach generates explanations very quickly, never exceeding half a second for even the largest BTs and states. Given the time it may take a user to produce a query and subsequently take in and process one or more explanations, such runtimes are certainly suitable for real-time robotics applications.

Regarding the number of nodes in the explanation model, we see a general trend in which the number of nodes grows with the number of BT nodes and the number of state variables, as predicted in Section 5.4.1. The variability present in Table 3 is due to the different seeds used for each parameter set and the different degrees of connectivity.

Similarly, the number of counterfactual candidates considered during the counterfactual search (Algorithm 6) also grows with the size of the BT and/or state, due to the larger explanation model over which the search is conducted. Once again, we observe high variability in the number of interventions considered, this time due to the position of the queried node in the explanation model. If the query targets a node with few ancestors, then much of the explanation model is discarded in the search, and thus even a very large explanation model could result in a small counterfactual search. The number of ancestors a node has in the explanation model depends on how far right the corresponding BT node is in the tree, as well as the “connectivity” and dynamics of the state.

Finally, we observe that the number of explanations in the explanation set also grows with the number of state variables, due to the larger number of counterfactual candidates which are considered. We see that the number of BT nodes does not seem to have much of an effect on the size of the explanation set, which we hypothesise is due to the *target recovery* metric itself, which explains the first difference between executions, which is more likely to occur earlier in the BT. We also note that, while the number of explanations returned is usually quite small (mean of 5.70 for the largest problem), outlier explanation sets can be quite large (maximum of 14 for the largest problem). In these cases, it is clear that further search filters, such as those discussed in Section 5.4, are required to produce manageable explanation sets. Ultimately, the size of the explanation set is heavily dependent on the size of the problem (i.e. the size of the explanation model), the variable queried, and the particular implementation of the functions in $\mathcal{F}$.

7.2. Example: Serial Recall Task

While the experiments in the previous section show the applicability of our method to a wide class of BT and state graph structures, we also demonstrate the same *target recovery* metric in a larger, more complex problem that more closely matches a real-world application. In this case, we model the serial recall use case. Serial recall is the ability to recall a novel sequence in order, and can be measured using memory tasks where participants are provided a sequence of “items” (e.g. verbally, visually, with physical objects, etc.) and must correctly recall the order of the sequence (Hurlstone, 2024). We model such a task where the sequence is provided by a social robot. The use of social robots in cognitive exercise and therapy is steadily emerging (Yuan, Klavon, Liu, Lopez and Zhao, 2021; Andriella, Torras, Abdelnour and Alenyà, 2022; Cavallaro, Perillo, Romano, Sebillio and Vitiello, 2024), and the approach we outline here seeks to emulate the complexity of the decision-making in these real world applications in simulation.

In our formulation, the robot can select the length of the sequence (i.e. the number of items) and the complexity (number of unique items). Once the user repeats back the sequence, it can evaluate the sequence and, depending on the response time and accuracy in repeating the sequence, can choose to continue to a new sequence, repeat the existing sequence, or end the entire exercise. In between showing the sequences, the robot can also select “social actions” such as encouraging or congratulating the user, or trying to recapture their attention. The task repeats until a maximum number of sequences is reached or the robot decides to end the task prematurely.

Apart from task state variables set by the system, the state also consists of a model of the user. Adapting the “persona” of Andriella, Torras and Alenyà (2019), we define a user along several dimensions, namely Attention, Memory, Reactivity and Frustration, each ranging from 0 (low) to 1 (high). These factors, together with the difficulty of the task, in turn affect other user variables such as Confusion and Engagement, as well as the response time and accuracy of the user when repeating a sequence.

We use a BT to implement the robot’s decision-making, and have it interact with a simulated user. We also construct a state model to represent causal assumptions about the user which correspond to rules in the simulation (e.g. such as the influence of user attention and confusion on engagement, or the influence of reactivity and sequence length and complexity on the user’s response time). Some noise is introduced to the user’s accuracy and response time to allow for differences between executions. The noise is controlled by a predetermined seed so as to not violate the assumptions that the behaviour and state are deterministic (see Section 4.3). After fully implementing the simulated use case, the resulting BT has 33 nodes (of which 18 are leaf nodes), and the state is represented as a configuration of 26 state variables, some discrete and some continuous (both are outlined more extensively in Appendix B). The resulting explanation model built by Algorithm 1 contains 177 nodes and 406 edges, demonstrating the complexity of this task.

To evaluate our approach on this use case, we again employ the *target recovery* metric. This time, we define a “default” user profile with initial values representing a typical user (setting Attention, Reactivity and Memory to 0.8 and Frustration to 0). The altered initialisations are generated by constructing alternate user profiles with a single perturbed user attribute (*Frustrated* sets Frustration to 1, *No Attention*, *No Reactivity* and *No Memory* set Attention, Reactivity and Memory to 0, respectively). Each profile is executed across 50 different seeds, and thus there are 50 comparisons between the default profile and each altered profile. The results of the *target recovery* metric, together

Profile	Number of Runs			TRR	Runtime (s)			No. Counterfactual Candidates			No. Explanations		
	Target	Found	No Difference		Min.	Max.	Mean (Std)	Min.	Max.	Mean (Std)	Min.	Max.	Mean (Std)
Frustrated	47	3	1	1	0.6752	1.3029	0.7122 (0.0891)	396	545	399.49 (21.69)	1	7	1.13 (0.88)
No Attention	50	0	1	1	0.0413	0.884	0.5464 (0.4033)	122	449	321.58 (159.37)	5	21	11.22 (4.35)
No Reactivity	49	1	1	1	0.041	0.3973	0.0806 (0.1070)	122	294	139.55 (52.60)	3	37	7.31 (9.89)
No Memory	50	0	1	1	0.6821	0.8961	0.7446 (0.0694)	396	449	406.60 (21.42)	4	37	18.64 (13.43)
All	196	4	1	1	0.041	1.3029	0.5203 (0.3425)	122	545	316.44 (137.50)	1	37	9.71 (10.70)

Table 4

Results for target recovery experiments in the serial recall use case. The behaviour tree has 33 nodes (18 leaves) and uses a state consisting of 26 variables. The resulting explanation model contains 177 nodes and 406 edges.

with runtime statistics, the number of counterfactual candidates considered and the number of explanations in the explanation set, are provided in Table 4.

In 196 (98%) of the comparisons, the perturbation resulted in a different in behaviour between the default and the altered profiles. And in every single case, our approach successfully recovered the target variable among the explanation set, demonstrating its correctness even for a more complex BT and state model. Despite the much larger size and complexity of the serial recall BT when compared to the randomly generated ones in Section 7.1, the time taken to generate explanations remains low enough for applicability in real-time robotics use cases.

Examining the number of counterfactual candidates and number of explanations returned for each altered profile, which are larger for this problem than the smaller randomly generated BTs as presented in Table 3, we see the dependence of the sizes of the search and explanation set on not only the size of the explanation model but also the part of the model that is queried. For example, the changes triggered by the *Frustrated* profile result in queries that produce relatively small explanation sets (mean 1.13), despite a relatively large number of counterfactual candidates considered. This is in part because the effects of the *Frustration* variable are limited, with only a single behaviour node having the variable in its input set (though one that occurs towards the right-most edge of the BT, hence the large number of counterfactual candidates). The changes triggered by the *No Memory* profile, on the other hand, result in much larger explanation sets (mean 18.64) and a greater search space (mean 406.60), with the *Memory* variable and its descendants impacting more nodes in the BT (5 nodes). The variance between explanation set sizes can also be explained by the difference in the implementations of each node (i.e. the functions in $\mathcal{F}$). Overall, we observe that a number of factors, including the size of the BT and state, the nature of the functions in $\mathcal{F}$, and the variable being queried, have the possibility of producing overly large sets of explanations in certain cases. If we are to reduce this set of explanations to a smaller, more relevant set, this notion of relevance must be encoded for, for example, by applying filters to the counterfactual search that account for properties such as discriminative power or actionability, as discussed in Section 8.

7.3. Comparison with an LLM Baseline

While most of the methods discussed in Section 3 cannot be directly compared to our method as they either address different types of queries (e.g. answering “what” questions instead of “why” questions) or can only provide a limited subset of causal explanations (as discussed in Section 6.2), approaches that leverage the “common sense” of LLMs are able to provide explanations in response to any natural language query and are thus suitable for comparison. Indeed, as discussed in Section 3, several works address the problem of LLM-based explanations for robots (Frering et al., 2025; Gebellí et al., 2025), including the work of Tagliamonte et al. (2024) which specifically targets behaviour trees. In this section, we examine the results of some variations of an LLM-based approach inspired by Tagliamonte et al. (2024) in the serial recall domain, and compare against the results obtained in Section 7.2.

To make the comparison as fair as possible, we give the LLM access to all the same information that our method uses to generate explanations. This includes textual descriptions of the task, the environment state (including all information contained in the state model), and BT (including its structure, input set, output set, action set, and detailed behavioural description), as well as the raw episodic memory. This information is fed to the LLM in the form of a system prompt¹, which also details the expected format of the explanation and examples of correct explanations (generated by our approach on different seeds than those tested in the evaluation).

We compare two versions of the prompt: a *simple* case, in which the LLM is instructed to provide only a single reason of the form $Var = X$, and a *complete* case, in which the LLM is instructed to provide multiple explanations of

¹The full prompts are available at https://anonymous.4open.science/r/btcm-AFE7/btcm/experiment/example_prompts/README.md

Model	Prompt	Number of Runs			TRR	Correct Reason		Incorrect Variable	Min.	Runtime (s)		
		Explained	No Difference	Error		Variable	Value			Max.	Mean (Std)	
phi4	Simple	190	4	6	0.06 (0.24)	0.04 (0.15)	0.22 (0.33)	188	0.9586	4.3452	2.3477 (0.5874)	
	Complete	195	4	1	0.20 (0.40)	0.22 (0.36)	0.35 (0.40)	161	2.356	13.1847	4.3336 (1.2712)	
deepseek-r1:32b	Simple	195	4	1	0.45 (0.50)	0.59 (0.48)	0.58 (0.49)	8	4.9088	95.2549	14.3402 (9.9605)	
	Complete	179	4	17	0.53 (0.50)	0.53 (0.31)	0.21 (0.28)	31	11.9767	65.0887	23.8133 (6.8669)	

Table 5

Results for evaluations performed in the serial recall domain with two LLMs with two sets of prompts. We display the number of runs in which explanations were successfully generated (*Explained*), the number of runs in which no difference in execution occurs (*No Difference*), and the number of runs in which a formatting error creates an unusable explanation (*Error*). We also display the *target recovery rate* (*TRR*), the ratio of explanations in which the identified reason variable is a viable causal explanation (*Variable*), the ratio of explanations which also correctly attributed the variable value from the episodic memory (*Value*), the number of runs in which a variable not present in the state model or BT is used in an explanation (*Incorrect*), and runtime statistics. Evaluations are performed on the same hardware, seeds and profiles as in Section 7.2, and thus results are directly comparable to those presented in Table 4.

the form $\langle \mathbf{R}, \langle \mathbf{J}, \mathbf{K} \rangle \rangle$, as defined in Section 4.2. For each version of the prompt, we compare two models: *phi4* (Abdin, Aneja, Behl, Bubeck et al., 2024) and *deepseek-r1:32b* (Guo, Yang, Zhang, Song et al., 2025), with 14B and 32B parameters respectively. Both models were run locally using Ollama², so that runtimes could be compared with those reported in the previous sections.

In addition to the *target recovery rate* (TRR) and runtime measured in previous sections, we also measure the rate at which the LLM identifies variables which are in the explanation set produced by our method (*correct reason variable*), the rate at which these correctly identified variables have the correct value according to the episodic memory (*correct reason value*), and the number of runs in which the LLM produced reasons using variables which do not exist in either the state model or the BT (*incorrect variable*). We also note the number of runs in which a formatting error renders the explanation unusable by the evaluation system. Results are collected over 200 runs (50 per profile), using the same seeds and profiles as in Section 7.2, and are tabulated in Table 5.

First of all, we note that the *target recovery rate* is low, even for the larger model. When compared to the 100% *target recovery rate* achieved by our method, the necessity for explicit causal modelling of the task becomes clear. Even though the LLMs have access to all the necessary information through the prompt, they lack the ability to causally reason and chain together causes and effects in a consistent way, which is essential for providing causal explanations, especially for temporal sequences of events where cause and effect may be separated in time. We also note that the larger model unsurprisingly performs significantly better, which holds for the remaining metrics (aside from runtime). We also note that the *complete* prompt results in improved *target recovery rate*, likely due to the LLM producing more explanations, which gives a higher probability of selecting the correct target variable.

Setting aside the *target recovery metric* and considering the complete possible set of correct explanations, we observe similar rates at which the LLMs identify correct reason variables in their explanation sets (*Correct Reason - Variable* in Table 5). These results indicate that the models, especially the smaller model, consistently identify reasons which are not causally correct (i.e. intervening on that variable would not result in satisfying the query when conducting the counterfactual search in Algorithm 6). Furthermore, when the LLMs do identify a correct reason variable, they very often attribute incorrect values in the reason (*Correct Reason - Variable* in Table 5), likely due to the difficulty in retrieving the appropriate values from a potentially very large episodic memory. Indeed, the generative approach taken by LLMs often results in so-called “hallucinations”, where responses are either incorrect or nonsensical (Huang, Yu, Ma, Zhong et al., 2025). This effect is most clear when considering the number of runs for which the LLMs referenced variables that did not exist in either the state model or the BT (*Incorrect Variable* in Table 5). This is especially a problem for the smaller model, which almost always included at least one “hallucinated” reason in the explanation set. In many cases, these incorrect variables may clearly reference a real variable (e.g. “UserEngagementLevel”, while incorrect, is very similar to the real variable “UserEngagement”), some are completely untethered to the state and BT information provided in the prompt (e.g. “CompletionStatus” or “SequenceEffectiveness”). Both models also occasionally produced explanations which disregarded formatting instructions, rendering the evaluation system unable to extract the reasons from the model output. The inconsistency in output formatting poses challenges to integrating an LLM-based explanation module into a larger architecture, and together with the problem of “hallucination”, casts further doubt on the applicability of LLMs in use cases where correct explanations are of special importance. Our

²<https://docs.ollama.com/>

method, on the other hand, consistently generates explanations which are grounded in the state model and BT structure and are always consistently formatted by design, making integration into larger architectures considerably simpler.

Finally, we note that the LLMs take orders of magnitude longer to produce explanations. In the best case (*phi4 - Simple*), the model runs on average 4.5 times slower than our method, while in the worst case (*deepseek-r1:32b - Complete*), the average runtime is 45.8 times slower. LLM explanation runtimes, in addition to being considerably slower than our method (the minimum runtime in Table 5 is within half a second of the maximum runtime of our method in Table 4), also suffer from high variance, rendering these models impractical for real-time integration into robotics systems.

To conclude this comparison, it is clear that an LLM-based approach is vastly inferior to our own method in terms of both explanation correctness and runtime, highlighting the need for causal reasoning in explanation generation.

8. Limitations and Future Work

While our proposed architecture for generating causal explanations from behaviour tree executions is capable of producing accurate explanations of decisions, executions, node returns and state values, expanding the scope of causal explanations for BTs from the related work identified in Section 3, it nevertheless carries limitations which should be addressed in future research.

Likely the biggest limitation of our approach is the assumption that the state and BT are completely deterministic, as discussed in Section 4.3. Such an assumption is required to ensure that explanations are consistent and correct, as the counterfactual search directly calls the functions in $\mathcal{F}$. Relaxing this assumption is an important next step for this research, and may require treating the values of nodes in the explanation model as probability distributions rather than deterministic function outputs, such as the causal Bayesian networks used by Diehl and Ramirez-Amaro (2022). In doing so, the extended architecture could account for uncertainty in both the inputs to explanation model functions (such as uncertain states caused by unobserved state variables or noisy sensors) and the outputs of said functions, allowing the architecture to handle stochastic functions within the behaviour tree and uncertain outcomes of decisions (e.g. due to failures in robot action executions).

Another important consideration is the need to inject domain knowledge in the form of a state model into the architecture in order to produce the final explanation model (see Section 5.2.2). Indeed, as demonstrated in Section 6.3, such domain knowledge is key to allowing the explanation model to recover certain target variables in its explanations. In this work, we remain agnostic to the source of the domain knowledge, focusing instead on its utility in constructing the explanation model. However, in the service of fully automating the explanation model construction, an important component of future work may be to investigate methods of automating the construction of the state model. As the state model encodes causal relations, its automatic construction requires causal discovery, which is an active research area that has produced a variety of methods (Wang, Huang, Wang, Liao, Li and Liu, 2024). Promising directions may be to extract the causal structure and functions of the state model from knowledge structures such as ontologies (Ben Messaoud, Leray and Ben Amor, 2009; Stegnar, Rožanec, Leban and Mladenović, 2024), or to use learning-based approaches from real-world data or simulation (Cagri, Mghames, Hanheide and Bellotto, 2023; Edström, Hellström and De Luna, 2023).

Furthermore, as discussed in Section 5.4.2 and as can be seen in Tables 2, 3 and 4, the size of the explanation set returned by the counterfactual search can vary significantly depending on the size and complexity of the explanation model, as determined by the BT and state model. While our approach guarantees that all valid, minimal counterfactuals are found, it may be useful to restrict this search to provide a smaller, more relevant set of explanations, as is generally preferred by users (Miller, 2019). This may be achieved by adding further restrictions during the counterfactual search, based on desirable properties such as *discriminative power* (as indicated by the critical influences discussed by Albini et al. (2020)) or *plausibility* (which could be based on data regarding the most probable state configurations). Further relevance criteria could be devised, such as using the distance between nodes in the explanation model, or by further creating stricter queries using contextual information and models of the user. Applying criteria to reduce the search space also has the effect of reducing the complexity of the counterfactual search, which, as noted in Section 5.4.2, is greatly impacted by the size of the explanation model. Additionally, LLMs represent a promising tool which could see applications in reducing or summarising an explanation set after the counterfactual search has been conducted (Dhaini, Erdogan, Bakshi and Kasneci, 2024).

In this work, our evaluation focuses on functionally-grounded approaches, excluding human- or application-grounded evaluations which require user studies conducted with human participants (Doshi-Velez and Kim, 2017). While user studies are necessary in order to evaluate the effect of explanations on desiderata such as understandability

and trust, we argue it is crucial to validate explanation generation functionally, in terms of properties such as correctness and runtime efficiency, before testing with users. Having performed such validations in this work, evaluating our approach in user studies becomes possible. Such user studies require explanation generation to be embedded into a larger human-robot interaction, requiring interfaces for providing queries and for communicating explanations (Anjomshoae et al., 2019; Matarese et al., 2021). Such interfaces require additional design decisions than the ones we have taken in this work to generate the content of explanations. Our architecture requires formal representations of queries and produces formal representations of explanations (as defined in Section 4.2). Leveraging LLMs to convert natural language questions into formal explanation queries, sensitive to question context within an environment and a broader human-robot interaction, may prove to be fruitful. Similarly, LLMs may be used to convert formal explanations or simple templates into natural language explanations, whose effectiveness in communicating information can be evaluated in studies with human participants. Aside from natural language, other modalities such as visual interfaces (e.g. annotated BTs or causal models) can be explored (Hoque and Mueller, 2021).

In our comparison with an LLM-based baseline in Section 7.3, we demonstrated that the LLMs produced, on average, less accurate explanations with much longer runtimes than our approach. One limitation of this comparison is that our baseline likely does not represent the best performance an LLM-based approach could achieve for explainability. In particular, our approach is end-to-end and monolithic, producing explanations with a single, large prompt. Better performance may be possible with a more “agentic” approach, in which functionality of the method is divided into a chain of LLM agents performing specific tasks (e.g. interpreting a query, identifying relevant parts of the episodic memory, chaining together causes and effects, synthesising an explanation set, etc.). However, we hypothesise that the problems identified with our LLM baseline, namely target recovery, the inclusion of incorrect information, and poor runtime performance, are likely to persist in such an approach, due to the “inevitability” of “hallucinations” (Xu et al., 2024). Of course, this does not preclude LLMs from being useful components of an explainability system. Indeed, areas which are not addressed by our method may be more suitable to LLMs, as discussed above. However, it is clear that the process of explanation generation, at least in terms of identifying reasons for BT behaviour and other events, is better left to our method, which produces consistent and correct explanations in a practical time frame.

Finally, further work could also be done to expand the range of decision-making structures from which an explanation model can be built. The class of BTs can be expanded by including other important node types, such as parallel nodes and decorators. Other common types of decision-making structures can also be explored, such as finite state machines (Iovino, Förster, Falco, Chung, Siegwart and Smith, 2024) and planning frameworks such as hierarchical task networks (Hayes and Scassellati, 2016).

9. Conclusion

In conclusion, we have presented a novel approach to generating counterfactual, causal explanations from BTs, leveraging a causal model automatically built from the BT structure and domain knowledge of the state and individual BT nodes. Our method is able to answer contrastive queries about node executions and return statuses, decisions made by a BT and the value of state variables in the environment. As the explanation model is built from the BT structure, state model and BT node inputs and outputs, our method is able to use all of these sources of information when generating explanations, and can take into account events earlier in the execution when producing explanations about later events. When compared to other methods, our method differs from model-agnostic approaches in its ability to leverage BT structure and domain knowledge to generate causal explanations. It differs from other BT-based approaches which do take advantage of BT structure in that it is able to answer a wide range of contrastive questions with reasons beyond a trace of executions. Finally, it differs from generative (i.e. LLM-based) approaches which are able to answer these types of questions in that the explanations are guaranteed to respect the causality of the system, with fidelity proportional to that of the injected domain knowledge.

Through evaluations in both a wide range of randomly generated problems and with a larger BT simulating a real HRI use case, we have demonstrated that the explanations generated by our approach are correct and can accurately recover the causes of differences in behaviour. Moreover, our evaluations have shown that these explanations can be generated in real time, suitable for robotics applications. This stands in contrast to LLM-based approaches which are not only more computationally demanding but are also inconsistent in their accuracy.

As discussed in Section 8, there are a number of lines of potential future that are open which may address the limitations of this work, particularly in the automatic learning of relevant domain knowledge, probabilistic approaches for handling stochasticity and uncertainty, and leveraging notions of actionability, plausability and relevance for

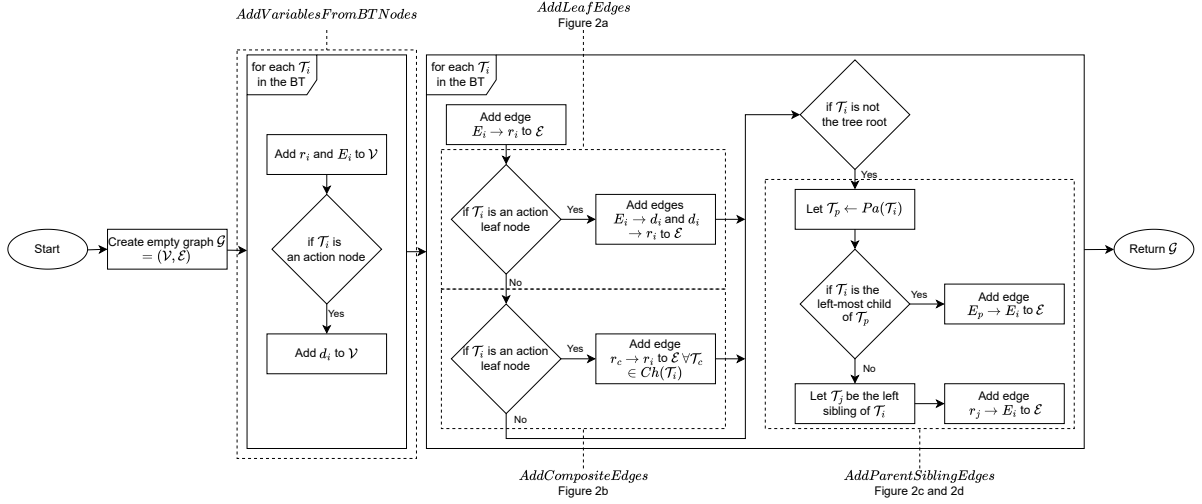


Figure 8: A flowchart depicting the process of constructing the explanation model from the BT structure and state model, as described in Section 5.2. In particular, the figure represents the procedure undertaken by Algorithm 1.

reducing the size of the counterfactual search space and/or explanation set. Further work can also be done to validate approaches built from this framework in user studies with real environments and physical robots, requiring the appropriate interfaces to be built for query formulation and explanation communication. Ultimately, these extensions require a solid foundation of accurate, causal explanations which are provided by our method.

In summary, not only is our method a robust approach to generating correct counterfactual explanations for BTs, but it also sets the foundation for the future development of transparent, interpretable robots capable of interactively and automatically explaining their decisions, which in turn has the potential to improve HRI by fostering increased understandability and trust.

Acknowledgements

Acknowledgement section anonymised for review.

A. Algorithms

In this appendix we expand on the algorithms presented in Section 5 with more detailed pseudocode. Algorithm 7 depicts the complete pseudocode of Algorithm 2, as described in Section 5.2.1. Algorithm 8 depicts the temporal linking function used in Algorithm 3, described in Section 5.2.2. Algorithm 9, which in turn makes use of Algorithms 10 - 13, describes the functions $\mathcal{F}$ of the nodes in the explanation graph $\mathcal{G}$, as described in Section 5.2.3.

In addition to pseudocode, we also present two flowcharts which provide a visual depiction of the key contributions of this work. The procedure for building the explanation model is depicted in Figure 8, while the procedure for performing the counterfactual search is depicted in Figure 9.

B. Serial Recall Task

To better contextualise the serial recall problem addressed in Section 7.2, we present a diagram of the full behaviour tree, generated by the *py_trees* library in Figure 10. The BT has 33 nodes, 18 of which are leaf nodes. We also provide a depiction of the state model of the environment used in the evaluation in Figure 11. The state model consists of the following variables:

- *EndGame* - a boolean flag denoting whether or not the robot should end the entire exercise
- *RepeatSequence* - a boolean flag denoting whether or not the robot should repeat the current sequence

Algorithm 7 Constructing the explanation graph from the behaviour tree structure - complete pseudocode

```

1: function GRAPHFROMSTRUCTURE( )
2:    $\mathcal{G} \leftarrow \emptyset$ 
3:    $\mathcal{G} \leftarrow \text{ADDVARIABLESFROMBTNODES}(\mathcal{G})$ 
4:   for node  $\mathcal{T}_i$  in the behaviour tree do
5:     Add edge  $E_i \rightarrow r_i$  to  $\mathcal{E}$ 
6:     if  $\mathcal{T}_i$  is a leaf node then
7:        $\mathcal{G} \leftarrow \text{ADDLEAFEDGES}(\mathcal{G}, \mathcal{T}_i)$ 
8:     else
9:        $\mathcal{G} \leftarrow \text{ADDCOMPOSITEEDGES}(\mathcal{G}, \mathcal{T}_i)$ 
10:    if  $\mathcal{T}_i$  is not the tree root then
11:       $\mathcal{G} \leftarrow \text{ADDPARENTSIBLINGEDGES}(\mathcal{G}, \mathcal{T}_i)$ 
12:   return  $\mathcal{G}$ 
13: function ADDVARIABLESFROMBTNODES( $\mathcal{G}$ )
14:   for node  $\mathcal{T}_i$  in the behaviour tree do
15:     Add variables  $r_i$  and  $E_i$  to  $\mathcal{V}$ 
16:     if  $\mathcal{T}_i$  is an action node then
17:       Add variable  $d_i$  to  $\mathcal{V}$ 
18:   return  $\mathcal{G}$ 
19: function ADDLEAFEDGES( $\mathcal{G}, L_i$ )
20:   if  $L_i$  is an action node then
21:     Add edge  $E_i \rightarrow d_i$  to  $\mathcal{E}$ 
22:     Add edge  $d_i \rightarrow r_i$  to  $\mathcal{E}$ 
23:   return  $\mathcal{G}$ 
24: function ADDCOMPOSITEEDGES( $\mathcal{G}, \mathcal{T}_i$ )
25:   for each child  $\mathcal{T}_c \in Ch(\mathcal{T}_i)$  do
26:     Add edge  $r_c \rightarrow r_i$  to  $\mathcal{E}$ 
27:   return  $\mathcal{G}$ 
28: function ADDPARENTSIBLINGEDGES( $\mathcal{G}, \mathcal{T}_i$ )
29:    $\mathcal{T}_p \leftarrow Pa(\mathcal{T}_i)$ 
30:   if  $\mathcal{T}_i$  is the left-most child of  $\mathcal{T}_p$  then
31:     Add edge  $E_p \rightarrow E_i$  to  $\mathcal{E}$ 
32:   else
33:     Let  $\mathcal{T}_j$  be the left sibling of  $\mathcal{T}_i$ 
34:     Add edge  $r_j \rightarrow E_i$  to  $\mathcal{E}$ 
35:   return  $\mathcal{G}$ 

```

- *SequenceComplexity* - the number of unique symbols in the sequence. From the set $\{2, 3, 4\}$
- *SequenceLength* - the number of symbols in the sequence. From the set $\{4, 5, 6, 7, 8\}$
- *NumRepetitions* - the number of times the current sequence has been shown to the user. From the set $\{0, 1, 2, 3\}$
- *NumSequences* - the total number of unique sequences shown to the user. From the set $\{0, 1, 2, 3\}$
- *SequenceSet* - a boolean flag denoting whether or not the robot has set the current sequence
- *ResponseTimerActive* - a boolean flag denoting whether or not the robot is waiting for the user to provide a response
- *AttemptedReengageUser* - a boolean flag denoting whether or not the robot has attempted to recapture the attention of the user for the current sequence

Algorithm 8 Linking temporal versions of state variables

```

1: function ADDTEMPORALEDGES( $\mathcal{G}, \tau$ )
2:   for  $X \in \mathcal{X}$  do
3:     if  $X$  has no parents in  $\mathcal{G}_S$  then
4:       for  $t \in [0, \tau(X))$  do
5:         Add edge  $X^{(t)} \rightarrow X^{(t+1)}$  to  $\mathcal{E}$ 
6:   return  $\mathcal{G}$ 

```

Algorithm 9 Defining $\mathcal{F}$ for each node in the explanation model

```

1: function  $\mathcal{F}_n(v)$ 
2:   if  $n \hat{=} E_i$  for some node  $\mathcal{T}_i$  then
3:     return EXECUTION( $\mathcal{T}_i$ )
4:   Let  $\mathcal{P} = \{p | p \text{ is a parent of } n \text{ in } \mathcal{G}\}$ 
5:    $s \leftarrow \text{SETINPUTSTATE}(n, v, \mathcal{P})$ 
6:   if  $n \hat{=} r_i$  for some node  $\mathcal{T}_i$  then
7:     return RETURN( $\mathcal{T}_i, s$ )
8:   else if  $n \hat{=} d_i$  for some leaf node  $L_i$  then
9:     return DECISION( $L_i, s$ )
10:  else
11:    if  $p \hat{=} x_k \in \mathcal{X} \forall p \in \mathcal{P}$  then
12:      return EXTERNALSTATE( $X^{(t)}, s$ )
13:    else
14:      return INTERNALSTATE( $X^{(t)}, s, v$ )
15: function SETINPUTSTATE( $n, v, \mathcal{P}$ )
16:   Let  $s$  be an assignment of variables in  $\mathcal{X}$ 
17:   for  $p \in \mathcal{P}$  do
18:     if  $p \hat{=} x_k \in \mathcal{X}$  then
19:       Set the value of  $x_k \in s$  to the value of  $p \in v$ 
20:   return  $s$ 

```

Algorithm 10 Defining $\mathcal{F}$ for execution nodes E_i

```

1: function EXECUTION( $\mathcal{T}_i$ )
2:   if  $\mathcal{T}_i$  is the root of the tree then
3:     return true
4:   else if  $\mathcal{T}_i$  is the left-most child of  $Pa(\mathcal{T}_i)$  then
5:      $\mathcal{T}_p \leftarrow Pa(\mathcal{T}_i)$ 
6:     return  $E_p$ 
7:   else
8:     Let  $\mathcal{T}_j$  be the sibling of  $\mathcal{T}_i$  immediately to its left
9:     if  $Pa(\mathcal{T}_i)$  is a sequence node then
10:      return true if  $r_j = \text{Success}$ , else false
11:     else if  $Pa(\mathcal{T}_i)$  is a fallback node then
12:      return true if  $r_j = \text{Failure}$ , else false

```

- *FeedbackGiven* - a boolean flag denoting whether or not the robot has provided feedback (e.g. a hint) to the user for the current repetition of the sequence
- *UserResponded* - a boolean flag denoting whether or not the user has provided a response for the current repetition of the sequence

Algorithm 11 Defining $\mathcal{F}$ for return status nodes r_i

```

1: function RETURN( $\mathcal{T}_i, \mathbf{s}$ )
2:   if  $E_i$  is false then
3:     return Invalid
4:   if  $\mathcal{T}_i$  is an action node then
5:     return  $r_i(d_i, \mathbf{s})$ 
6:   else if  $\mathcal{T}_i$  is a condition node then
7:     return  $r_i(\mathbf{s})$ 
8:   else if  $\mathcal{T}_i$  is a sequence node then
9:     for  $\mathcal{T}_c \in Ch(\mathcal{T}_i)$  do
10:      if  $r_c \in \{Running, Failure, Invalid\}$  then
11:        return  $r_c$ 
12:      return Success
13:   else if  $\mathcal{T}_i$  is a fallback node then
14:     for  $\mathcal{T}_c \in Ch(\mathcal{T}_i)$  do
15:       if  $r_c \in \{Running, Success, Invalid\}$  then
16:         return  $r_c$ 
17:     return Failure

```

Algorithm 12 Defining $\mathcal{F}$ for decision nodes d_i

```

1: function DECISION( $L_i, \mathbf{s}$ )
2:   if  $E_i$  is false then
3:     return  $\emptyset$ 
4:   else
5:     return  $d_i(\mathbf{s})$ 

```

Algorithm 13 Defining $\mathcal{F}$ for state nodes $X^{(t)}$

```

1: function EXTERNALSTATE( $X^{(t)}, \mathbf{s}$ )
2:   Let  $X \in \mathcal{X}, X^{(t)} \hat{=} X$ 
3:   return  $\mathcal{F}_{SX}(\mathbf{s})$ 
4: function INTERNALSTATE( $X^{(t)}, \mathbf{s}, \mathbf{v}$ )
5:    $\exists L_i$  s.t.  $E_i, d_i$  are parents of  $X^{(t)}$  in  $\mathcal{G}$ 
6:   if  $E_i$  is false then
7:     return  $X^{(t-1)}$ 
8:   else
9:     Let  $\mathcal{P} = \{p | p \text{ is a parent of } d_i \text{ in } \mathcal{G}\}$ 
10:     $\mathbf{z} \leftarrow \text{SETINPUTSTATE}(d_i, \mathbf{v}, \mathcal{P})$ 
11:     $\mathbf{z}^* \leftarrow p_i(\mathbf{z})$ 
12:    Let  $X \in \mathcal{X}, X^{(t)} \hat{=} X$ 
13:    return value of  $X \in \mathbf{z}$ 

```

- *UserTimeout* - a boolean flag denoting whether or not the user has provided a response within the time limit of 7 seconds
- *CurrentSequence* - a string of *SequenceLength* symbols, composed of *SequenceComplexity* unique symbols. May not be intervened on by the explanation model directly
- *UserSequence* - the user's attempt to recall the sequence of symbols. May not be intervened on by the explanation model directly

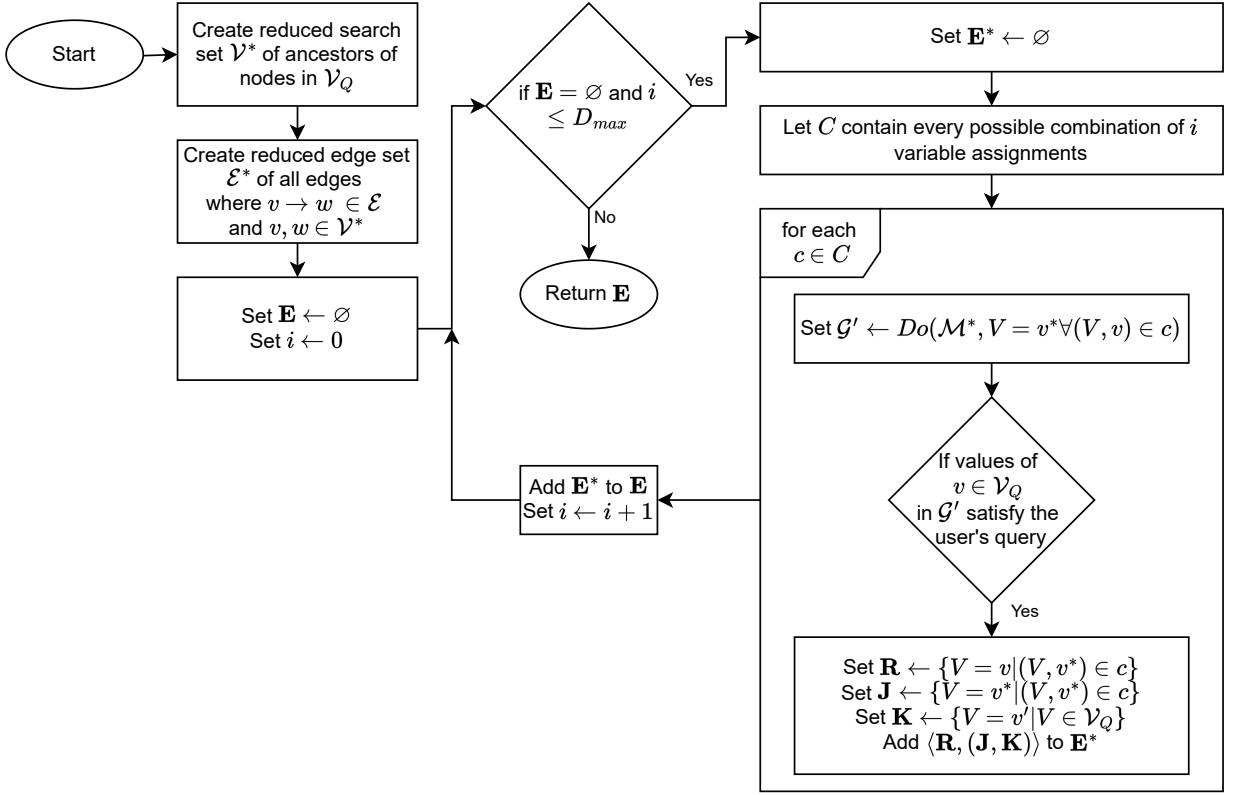


Figure 9: A flowchart depicting the counterfactual search, which performs interventions on the explanation model in order to build a set of causal explanations, as described in Section 5.3 and presented in Algorithm 6.

- *AccuracySeed* - a seed used to add noise to the user's accuracy and thus their response. May not be intervened on by the explanation model directly, and is included to satisfy deterministic assumption
- *ResponseTimeSeed* - a seed used to add noise to the user's response time. May not be intervened on by the explanation model directly, and is included to satisfy deterministic assumption
- *UserMemory* - a variable on the interval $[0, 1]$ denoting the ability for the user to recall information
- *UserAttention* - a variable on the interval $[0, 1]$ denoting the ability for the user to pay attention to events
- *UserReactivity* - a variable on the interval $[0, 1]$ denoting the reaction speed of the user
- *UserConfusion* - a variable on the interval $[0, 1]$ denoting how confused a user is in a given situation
- *UserEngagement* - a variable on the interval $[0, 1]$ denoting how engaged the user is with the task
- *UserFrustration* - a variable on the interval $[0, 1]$ denoting how frustrated the user is
- *BaseUserAccuracy* - a variable on the interval $[0, 1]$ denoting the accuracy of the user in recalling the correct sequence
- *UserNumErrors* - the number of errors a user makes for the current sequence, on the set $\{0, 1, 2, 3\}$
- *BaseUserResponseTime* - the number of seconds, before noise is added, that it takes for a user to respond, on the interval $[0, 7]$

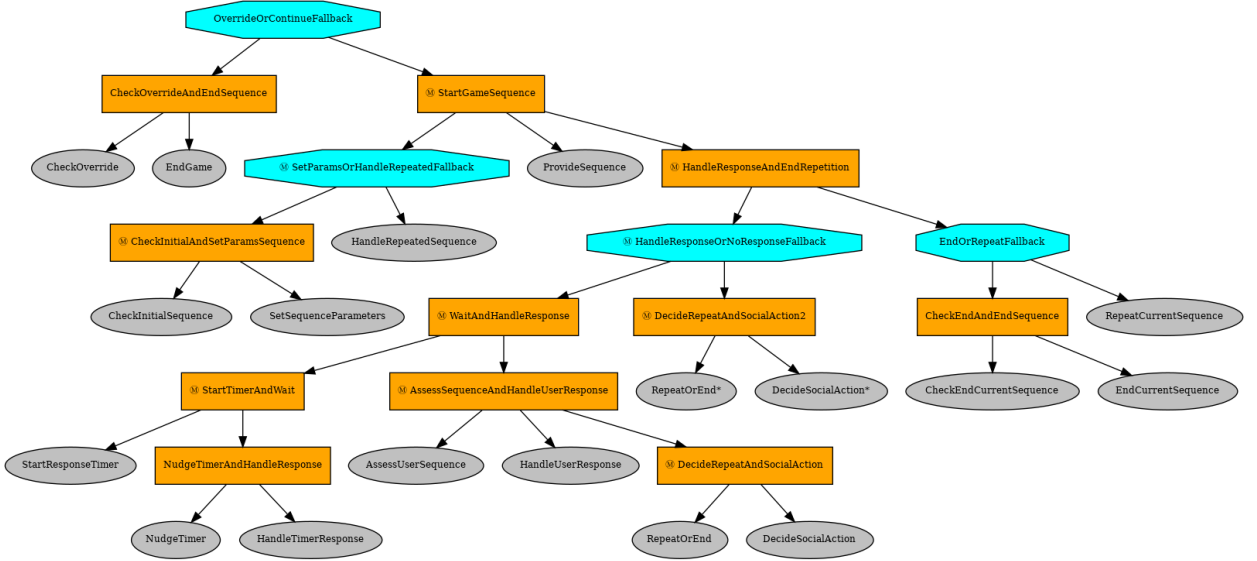


Figure 10: The full BT of the serial recall problem presented in Section 7.2. The BT has 33 nodes, 18 of which are leaf nodes. Following the visualisation standards of the *py_trees* library, fallback nodes are represented as light blue octagons, sequence nodes as orange rectangles, and leaf nodes as grey ovals.

- *ObservedUserResponseTime* - the number of seconds, after noise is added, that it takes for a user to respond, on the interval $[0, 7]$

The input, output and action sets (where applicable) are given below for each of the leaf nodes in the BT:

- *CheckOverride*
 - $\mathcal{X}_i = \{EndGame\}$
- *EndGame*
 - $\mathcal{A}_i = \{EndGameAction, \emptyset\}$
- *CheckInitialSequence*
 - $\mathcal{X}_i = \{NumRepetitions\}$
- *SetSequenceParameters*
 - $\mathcal{X}_i = \{NumSequences, UserMemory, UserAttention, UserReactivity, UserConfusion, UserEngagement, UserNumErrors, UserTimeout, SequenceComplexity, SequenceLength, ObservedUserResponseTime\}$
 - $\mathcal{Y}_i = \{SequenceLength, SequenceComplexity, SequenceSet, NumSequences, CurrentSequence\}$
 - $\mathcal{A}_i = \{SetSequenceParametersAction(L, C) | L \in \{4, 5, 6, 7, 8\}, C \in \{2, 3, 4\}\} \cup \{\emptyset\}$
- *HandleRepeatedSequence*
 - $\mathcal{X}_i = \{SequenceSet\}$
 - $\mathcal{A}_i = \{CrashAction, \emptyset\}$
- *ProvideSequence*
 - $\mathcal{X}_i = \{SequenceSet, NumRepetitions, SequenceLength, SequenceComplexity, UserMemory, UserAttention, UserReactivity, AccuracySeed, ResponseTimeSeed\}$

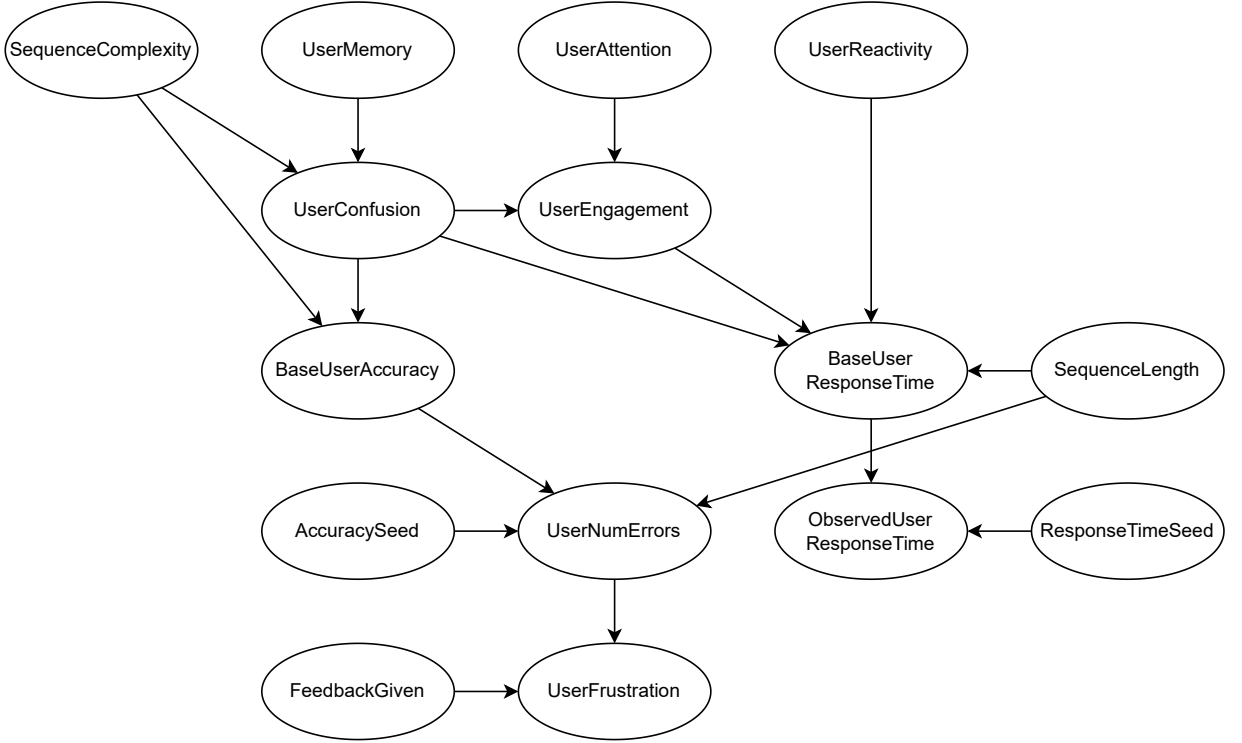


Figure 11: The state model $\mathcal{M}_S$ representing the injected domain knowledge in the form of a causal model. Variables listed in Appendix B but not included in the figure are not causally connected to any other variables (other than through interactions with the BT, as captured by the explanation model).

- $\mathcal{Y}_i = \{NumRepetitions, UserResponded, UserConfusion, UserEngagement, BaseUserAccuracy, UserNumErrors, BaseUserResponseTime, ObservedUserResponseTime, UserSequence, AccuracySeed, ResponseTimeSeed\}$
- $\mathcal{A}_i = \{ProvideSequenceAction, \emptyset\}$

- *StartResponseTimer*

- $\mathcal{X}_i = \{ResponseTimerActive\}$
- $\mathcal{Y}_i = \{ResponseTimerActive, UserResponded, UserTimeout\}$
- $\mathcal{A}_i = \{ResetTimerAction, \emptyset\}$

- *NudgeTimer*

- $\mathcal{X}_i = \{ObservedUserResponseTime\}$
- $\mathcal{Y}_i = \{UserResponded, UserTimeout\}$
- $\mathcal{A}_i = \{CheckTimerAction, \emptyset\}$

- *HandleTimerResponse*

- $\mathcal{X}_i = \{UserResponded, UserTimeout\}$
- $\mathcal{Y}_i = \{ResponseTimerActive\}$

- *AssessUserSequence*

- $\mathcal{A}_i = \{AssessSequenceAction, \emptyset\}$

- *HandleUserResponse*
 - $\mathcal{X}_i = \{UserResponded\}$
- *RepeatOrEnd*
 - $\mathcal{X}_i = \{NumRepetitions, UserResponded, UserFrustration, UserEngagement, AttemptedReengageUser, UserNumErrors\}$
 - $\mathcal{Y}_i = \{RepeatSequence\}$
 - $\mathcal{A}_i = \{RepeatThisSequenceAction, EndThisSequenceAction, \emptyset\}$
- *DecideSocialAction*
 - $\mathcal{X}_i = \{UserResponded, RepeatSequence, UserConfusion, UserEngagement, AttemptedReengageUser\}$
 - $\mathcal{Y}_i = \{AttemptedReengageUser, FeedbackGiven\}$
 - $\mathcal{A}_i = \{GiveSequenceHintAction, RepeatSequenceSocialAction, EndSequenceSocialAction, RecaptureAttention, \emptyset\}$
- *CheckEndCurrentSequence*
 - $\mathcal{X}_i = \{RepeatSequence\}$
- *EndCurrentSequence*
 - $\mathcal{X}_i = \{NumSequences\}$
 - $\mathcal{Y}_i = \{NumRepetitions, SequenceSet, ResponseTimerActive, AttemptedReengageUser, RepeatSequence, EndGame, FeedbackGiven\}$
 - $\mathcal{A}_i = \{EndThisSequenceAction, \emptyset\}$
- *RepeatCurrentSequence*
 - $\mathcal{A}_i = \{RepeatThisSequenceAction, \emptyset\}$

References

- Abdin, M., Aneja, J., Behl, H., Bubeck, S., et al., 2024. Phi-4 technical report. arXiv preprint arXiv:2412.08905 .
- Albini, E., Rago, A., Baroni, P., Toni, F., et al., 2020. Relation-based counterfactual explanations for Bayesian network classifiers, in: International Joint Conference on Artificial Intelligence, pp. 451–457. doi:10.5555/3491440.3491503.
- Andriella, A., Torras, C., Abdellour, C., Alenyà, G., 2022. Introducing CARESSER: A framework for in situ learning robot social assistance from expert knowledge and demonstrations. User Modeling and User-Adapted Interaction 33, 441. doi:10.1007/s11257-021-09316-5.
- Andriella, A., Torras, C., Alenyà, G., 2019. Learning robot policies using a high-level abstraction persona-behaviour simulator, in: International Conference on Robot and Human Interactive Communication, IEEE. pp. 1–8. doi:10.1109/RO-MAN46459.2019.8956357.
- Angelopoulos, G., Mangiacapra, L., Rossi, A., Di Napoli, C., Rossi, S., 2025. What is behind the curtain? Increasing transparency in reinforcement learning with human preferences and explanations. Engineering Applications of Artificial Intelligence 149, 110520. doi:10.1016/j.engappai.2025.110520.
- Anjomshoe, S., Najjar, A., Calvaresi, D., Främling, K., 2019. Explainable agents and robots: Results from a systematic literature review, in: International Conference on Autonomous Agents and Multiagent Systems (AAMAS), International Foundation for Autonomous Agents and Multiagent Systems. pp. 1078–1088.
- Barkouki, T.H., Chuang, I.T., Robinson, S.K., 2024. “What will you do next?” Designing and evaluating explanation generation using behavior trees for projection-level XAI, in: Companion of the International Conference on Human-Robot Interaction, ACM/IEEE. pp. 223–227. doi:10.1145/3610978.3640547.
- Ben Messaoud, M., Leray, P., Ben Amor, N., 2009. Integrating ontological knowledge for iterative causal discovery and visualization, in: European Conference on Symbolic and Quantitative Approaches to Reasoning and Uncertainty, Springer. pp. 168–179. doi:10.1007/978-3-642-02906-6_16.
- Castri, L., Mghames, S., Hanheide, M., Bellotto, N., 2023. Enhancing causal discovery from robot sensor data in dynamic scenarios, in: Conference on Causal Learning and Reasoning, PMLR. pp. 243–258.
- Cavallaro, A., Perillo, F., Romano, M., Sebillio, M., Vitiello, G., 2024. Social robot in service of the cognitive therapy of elderly people: Exploring robot acceptance in a real-world scenario. Image and Vision Computing 147, 105072. doi:10.1016/j.imavis.2024.105072.

- Colledanchise, M., Natale, L., 2021. On the implementation of behavior trees in robotics. *Robotics and Automation Letters* 6, 5929–5936. doi:10.1109/LRA.2021.3087442.
- Colledanchise, M., Ögren, P., 2014. How behavior trees modularize robustness and safety in hybrid systems, in: *International Conference on Intelligent Robots and Systems, IEEE/RSJ*. pp. 1482–1488. doi:10.1109/IRoS.2014.6942752.
- Colledanchise, M., Ögren, P., 2018. *Behavior trees in robotics and AI: An introduction*. CRC Press. doi:10.1201/9780429489105.
- Cruz, F., Dazeley, R., Vamplew, P., Moreira, I., 2023. Explainable robotic systems: Understanding goal-driven actions in a reinforcement learning scenario. *Neural Computing and Applications* 35, 18113–18130. doi:10.1007/s00521-021-06425-5.
- DeChant, C., 2023. On the risks and benefits of episodic memory in AI agents, in: *International Conference on Logic Programming: Workshop on Safe and Trustworthy AI*, pp. 1–6. URL: <https://www.cs.columbia.edu/~dechant/safeaiworkshop2023.pdf>.
- Dhaini, M., Erdogan, E., Bakshi, S., Kasneci, G., 2024. Explainability meets text summarization: A survey, in: *International Natural Language Generation Conference*, pp. 631–645. doi:10.18653/v1/2024.inlg-main.49.
- Diehl, M., Ramirez-Amaro, K., 2022. Why did I fail? A causal-based method to find explanations for robot failures. *Robotics and Automation Letters* 7, 8925–8932. doi:10.1109/LRA.2022.3188889.
- Diehl, M., Ramirez-Amaro, K., 2023. A causal-based approach to explain, predict and prevent failures in robotic tasks. *Robotics and Autonomous Systems* 162, 104376. doi:10.1016/j.robot.2023.104376.
- Doshi-Velez, F., Kim, B., 2017. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608* doi:10.48550/arXiv.1702.08608.
- Edström, F., Hellström, T., De Luna, X., 2023. Robot causal discovery aided by human interaction, in: *International Conference on Robot and Human Interactive Communication (RO-MAN)*, IEEE. pp. 1731–1736. doi:10.1109/RO-MAN57019.2023.10309376.
- Frering, L., Steinbauer-Wagner, G., Holzinger, A., 2025. Integrating belief-desire-intention agents with large language models for reliable human-robot interaction and explainable artificial intelligence. *Engineering Applications of Artificial Intelligence* 141, 109771. doi:10.1016/j.engappai.2024.109771.
- Gavriilidis, K., Munafo, A., Pang, W., Hastie, H., 2023. A surrogate model framework for explainable autonomous behaviour. *arXiv preprint arXiv:2305.19724* doi:10.48550/arXiv.2305.19724.
- Gebellí, F., Hriscu, L., Ros, R., Lemaignan, S., Sanfeliu, A., Garrell, A., 2025. Personalised explainable robots using LLMs, in: *International Conference on Human-Robot Interaction, ACM/IEEE*. pp. 1304–1308. doi:10.1109/HRI61500.2025.10974125.
- Gjærum, V.B., Strümke, I., Lekkas, A.M., Miller, T., 2023. Real-time counterfactual explanations for robotic systems with multiple continuous outputs. *IFAC-PapersOnLine* 56, 7–12. doi:10.1016/j.ifacol.2023.10.1328.
- Gómez-Talal, I., Azizoltani, M., Bote-Curiel, L., Rojo-Álvarez, J.L., Singh, A., 2025. Towards explainable artificial intelligence in machine learning: A study on efficient perturbation-based explanations. *Engineering Applications of Artificial Intelligence* 155, 110664. doi:10.1016/j.engappai.2025.110664.
- Gugliermo, S., Domínguez, D.C., Iannotta, M., Stoyanov, T., Schaffernicht, E., 2024. Evaluating behavior trees. *Robotics and Autonomous Systems* 178, 104714. doi:10.1016/j.robot.2024.104714.
- Guidotti, R., 2022. Counterfactual explanations and how to find them: literature review and benchmarking. *Data Mining and Knowledge Discovery*, 1–55. doi:10.1007/s10618-022-00831-6.
- Guo, D., Yang, D., Zhang, H., Song, J., et al., 2025. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Han, Z., Giger, D., Allspaw, J., Lee, M.S., Admoni, H., Yanco, H.A., 2021. Building the foundation of robot explanation generation using behavior trees. *Transactions on Human-Robot Interaction* 10, 1–31. doi:10.1145/3457185.
- Hayes, B., Scassellati, B., 2016. Autonomously constructing hierarchical task networks for planning and human-robot collaboration, in: *2016 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE. pp. 5469–5476. doi:10.1109/ICRA.2016.7487760.
- Herrera-Poyatos, A., Del Ser, J., de Prado, M.L., Wang, F.Y., et al., 2025. Responsible artificial intelligence systems: A roadmap to society's trust through trustworthy AI, auditability, accountability, and governance. *arXiv preprint arXiv:2503.04739*.
- Hoque, M.N., Mueller, K., 2021. Outcome-explorer: A causality guided interactive visual interface for interpretable algorithmic decision making. *IEEE Transactions on Visualization and Computer Graphics* 28, 4728–4740. doi:10.1109/TVCG.2021.3102051.
- Huang, L., Yu, W., Ma, W., Zhong, W., et al., 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 1–55. doi:10.1145/3703155.
- Hurlstone, M.J., 2024. Serial recall, in: *The Oxford Handbook of Human Memory, Two Volume Pack: Foundations and Applications*. Oxford University Press. chapter 28, pp. 799–830. doi:10.1093/oxfordhb/9780190917982.001.0001.
- Iovino, M., Förster, J., Falco, P., Chung, J.J., Siegwart, R., Smith, C., 2024. Comparison between behavior trees and finite state machines. *arXiv preprint arXiv:2405.16137* doi:10.48550/arXiv.2405.16137.
- Iovino, M., Scukins, E., Styrd, J., Ögren, P., Smith, C., 2022. A survey of behavior trees in robotics and AI. *Robotics and Autonomous Systems* 154, 104096. doi:10.1016/j.robot.2022.104096.
- Kanazawa, K., Koller, D., Russell, S., 1995. Stochastic simulation algorithms for dynamic probabilistic networks, in: *Conference on Uncertainty in Artificial Intelligence*, pp. 346–351. doi:10.5555/2074158.2074197.
- Langley, P., Meadows, B., Sridharan, M., Choi, D., 2017. Explainable agency for intelligent autonomous systems, in: *Conference on Artificial Intelligence, AAAI*. pp. 4762–4763. doi:10.1609/aaai.v31i2.19108.
- LeMasurier, G., Gautam, A., Han, Z., Crandall, J.W., Yanco, H.A., 2024a. Reactive or proactive? How robots should explain failures, in: *International Conference on Human-Robot Interaction, ACM/IEEE*. pp. 413–422. doi:10.1145/3610977.3634963.
- LeMasurier, G., Tagliamonte, C., Breen, J., Maccaline, D., Yanco, H.A., 2024b. Templated vs. generative: Explaining robot failures, in: *International Conference on Robot and Human Interactive Communication, IEEE*. pp. 1346–1353. doi:10.1109/RO-MAN60168.2024.10731331.
- Love, T., Andriella, A., Alenyà, G., 2024. What would I do if...? Promoting understanding in HRI through real-time explanations in the wild, in: *International Conference on Robot and Human Interactive Communication, IEEE*. pp. 504–509. doi:10.1109/RO-MAN60168.2024.10731403.

- Matarese, M., Rea, F., Sciutti, A., 2021. A user-centred framework for explainable artificial intelligence in human-robot interaction. arXiv preprint arXiv:2109.12912 .
- Miller, T., 2019. Explanation in artificial intelligence: Insights from the social sciences. *Artificial intelligence* 267, 1–38. doi:10.1016/j.artint.2018.07.007.
- Miller, T., 2021. Contrastive explanation: A structural-model approach. *The Knowledge Engineering Review* 36, e14. doi:10.1017/S0269888921000102.
- Nauta, M., Trienes, J., Pathak, S., Nguyen, E., Peters, M., Schmitt, Y., Schlötterer, J., Van Keulen, M., Seifert, C., 2023. From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI. *Computing Surveys* 55, 1–42. doi:10.1145/3583558.
- Ögren, P., Alfredson, J., 2023. Creating trustworthy AI for UAS using labeled backchained behavior trees, in: *International Conference on Unmanned Aircraft Systems (ICUAS)*, IEEE. pp. 1029–1036. doi:10.1109/ICUAS57906.2023.10156149.
- Pearl, J., Mackenzie, D., 2018. *The book of why: the new science of cause and effect*. Basic books.
- Potteiger, N., Koutsoukos, X., 2023. Safe explainable agents for autonomous navigation using evolving behavior trees, in: *International Conference on Assured Autonomy*, IEEE. pp. 44–52. doi:10.1109/ICAA58325.2023.00014.
- Russell, S., Norvig, P., 2009. *Artificial Intelligence: A Modern Approach*. 3rd ed., Prentice Hall Press, USA.
- Sado, F., Loo, C.K., Liew, W.S., Kerzel, M., Wermter, S., 2023. Explainable goal-driven agents and robots-a comprehensive review. *Computing Surveys* 55, 1–41. doi:10.1145/3564240.
- Stegnar, J., Rožanec, J.M., Leban, G., Mladenčić, D., 2024. Enhancing causal graphs with domain knowledge: Matching ontology concepts between ontologies and raw text data. *Proceedings of the Information Society* .
- Stonier, D., Demir, E.M., Walker, N., Castro, S., et al., 2025. Pytrees 2.3.0. URL: <https://py-trees.readthedocs.io/en/release-2.3.x/>.
- Tagliamonte, C., Maccaline, D., LeMasurier, G., Yanco, H.A., 2024. A generalizable architecture for explaining robot failures using behavior trees and large language models, in: *Companion of the International Conference on Human-Robot Interaction*, ACM/IEEE. pp. 1038–1042. doi:10.1145/3610978.3640551.
- Wachowiak, L., Fenn, A., Kamran, H., Coles, A., Celiktutan, O., Canal, G., 2024. When do people want an explanation from a robot?, in: *International Conference on Human-Robot Interaction*, ACM/IEEE. pp. 752–761. doi:10.1145/3610977.3634990.
- Walker, P.B., Haase, J.J., Mehalick, M.L., Steele, C.T., et al., 2025. Harnessing metacognition for safe and responsible ai. *Technologies* 13, 107. doi:10.3390/technologies13030107.
- Wan, K., Liu, Y., Liu, H., Xu, X., 2024. Unraveling explainable reinforcement learning using behavior tree structures, in: *International Conference on Acoustics, Speech and Signal Processing*, IEEE. pp. 6465–6469. doi:10.1109/ICASSP48485.2024.10446357.
- Wang, L., Huang, S., Wang, S., Liao, J., Li, T., Liu, L., 2024. A survey of causal discovery based on functional causal model. *Engineering Applications of Artificial Intelligence* 133, 108258. doi:10.1016/j.engappai.2024.108258.
- Xu, Z., Jain, S., Kankanhalli, M., 2024. Hallucination is inevitable: An innate limitation of large language models. arXiv preprint arXiv:2401.11817 .
- Yuan, F., Klavon, E., Liu, Z., Lopez, R.P., Zhao, X., 2021. A systematic review of robotic rehabilitation for cognitive training. *Frontiers in Robotics and AI* 8, 605715. doi:10.3389/frobt.2021.605715.